

BernatLab

Hort Osona en acció: implementar el BernatLab pas a pas

Raspberry Pi · Docker · IA · LoRa · Hort Osona

Mòdul 7

Bernat — 8 de juliol del 2026

Document tècnic de consulta personal

Sobre aquest manual

Aquest és el segon mòdul del manual tècnic del BernatLab. Cobreix tota la cadena de sensors i visualització: el protocol MQTT, el broker Mosquitto, l'esquema de publicació dels sensors, la base de dades InfluxDB, l'agent Telegraf, la programació visual amb Node-RED, la visualització amb Grafana, l'API REST amb FastAPI, la integració amb la web pública Hort Osona i l'operativa del dia a dia.

Com es llegeix

En ordre, si parteixes de zero. Per capítols, si ja tens una base i vols aprofundir. Cada capítol segueix la mateixa estructura: teoria, aplicació al BernatLab, esquemes, comandes útils, errors habituals, exercicis pràctics i resum final.

Índex de capítols

- 11. Del Mòdul 1 al M2: què construïm
- 12. MQTT des de zero
- 13. Mosquitto al BernatLab
- 14. Publicar dades: els sensors
- 15. InfluxDB: base de dades de sèries temporals
- 16. Telegraf: el pont
- 17. Node-RED: programació visual
- 18. Fluxos pràctics
- 19. Grafana: visualitzar les dades
- 20. API pública: servir les dades al món
- 21. Integració amb Hort Osona
- 22. Operativa: còpies, alertes, escalat

Pròleg — Com s'ha fet aquest llibre

*“Si una eina és bona per ensenyar, també és bona per ensenyar-se a fer servir.”**

Aquest pròleg és una confessió. Explica com s'ha generat el llibre que tens a les mans, per què, i quines limitacions té. És tan pedagògic com la resta, perquè la millor manera d'aprendre a fer servir la IA local és veure com s'ha fet servir de veritat.

Les eines

Tres peces:

- **Ollama** al Mac (no pas a la Raspberry). És el motor que executa el model de llenguatge local. Si llegeixes el M4, ja saps què és.
- **Hermes** com a entorn de treball. Un assistent d'escriptura i programació que connecta amb el model i permet iterar, validar, i executar coses.
- **MiniMax-M3:Cloud** com a model de raonament. És el "cervell" que ha escrit la majoria dels capítols. La part de "Cloud" al nom vol dir que el model corre al núvol del proveïdor (no pas local al Mac), tot i que el plantejament del BernatLab defensa la IA local — i això és una de les limitacions que explico més avall.

El procés

El flux ha estat iteratiu, no pas lineal:

1. **Planificació:** l'autor (Bernat Mora) decidia el temari, l'estructura per mòduls, els capítols concrets. Això ho feia jo (l'assistent), proposant opcions i ell triant.
2. **Escriptura:** jo escrivia cada capítol seguint les 8 seccions estàndard (teoria, aplicació al BernatLab, esquemes, comandes, "què està passant realment", errors habituals, exercicis, resum). En una sola passada, sense revisions llargues.
3. **Validació amb execució real:** aquí ve la part important. Jo generava, però l'autor validava. Per exemple, jo proposava la instal·lació de Grafana; ell provava i em deia "el port 3000 no és correcte, és el 3000 sí" o "la comanda X falla perquè Debian 13 ja no té /var/log/auth.log". Jo corregia i re-generava.
4. **Iteració sobre artefactes:** PDF, DOCX, HTML. Cada vegada que canviava un capítol, jo regenerava tots els artefactes amb el meu propi `make_book.py` (sí, el codi que genera el llibre està al llibre — un bucle bonic).
5. **Publicació:** Git, GitHub Pages, commit, push. Tot automàtic.

El que la IA fa bé en aquest procés

- **Velocitat:** escriure 562 pàgines en poques setmanes seria impossible a mà.
- **Densitat:** jo no tinc la temptació d'omplir pàgines amb coses evidents. Si una secció és buida, s'elimina.

- **Replicabilitat:** el procés és 100% reproduïble. Si vols fer el teu propi llibre tècnic amb IA, ja tens la recepta.
- **Coherència:** el to, l'estructura, i el nivell de profunditat es mantenen al llarg de 69 capítols. Un humà sol esgotar-se o distreure's.

El que la IA NO fa bé

Aquí ve la part important, i honesta:

- **No tinc experiència viscuda.** Puc descriure com configurar Tailscale, però no puc explicar què se sent quan t'estàs 30 minuts barallant-te amb un acl que no rutlla. Aquestes coses les afegia l'autor (Bernat) quan les trobava, marcades com a "■■■ Pendent de validar" o afegides a peu de capítol.
- **Invento dates, números i detalls que semblen precisos però no ho són.** Tots els preus, les mides, els temps d'instal·lació d'aquest llibre són estimacions raonables, no pas mesures. Si tu vals més o menys, és normal.
- **No he provat res a la teva Raspberry.** Cap dels 12 capítols del M7 ha estat executat per mi a la teva mquina. L'autor els anirà validant a casa, i quan trobi diferències, s'actualitzaran.
- **Tinc biaixos de coneixement.** Soc bo explicant tecnologies populars (Docker, MQTT, Grafana), però menys fi en temes molt nous o molt específics. Si una explicació et sembla superficial, probablement és per això.
- **Cometo errors en el codi.** El codi YAML, Python, bash que surt al llibre és correcte en el 95% dels casos, però el 5% restant té fallades (versions obsoletes, comes que no toca, imports que no existeixen). Validar sempre.

Les limitacions del model

MiniMax-M3:Cloud és bo, però té limitacions:

- **No té memòria entre sessions.** Cada vegada que l'autor obria una nova sessió, jo no recordava res del que havíem parlat abans. Tots els detalls del projecte (IPs, configuracions, decisions preses) estaven en fitxers que jo llegia cada vegada.
- **No tinc accés directe a la Raspberry.** Tots els exemples que dono són teòrics o basats en documentació general. L'autor els validava a mà.
- **El nom del model "M3" ve de la família MiniMax M-series.** Són models de propòsit general, no pas entrenats específicament per a sistemes. Per tant, de tant en tant, es comporten de maneres inesperades.
- **El "Cloud" al nom vol dir que les meves respostes viatgen al núvol.** Tots els prompts i respostes passen per servidors externs. Per tant, **no és una solució d'IA local** malgrat que el plantejament del BernatLab defensa la privadesa. Per a tasques sensibles (claus SSH, contrasenyes, configuracions crítiques), l'autor mai no ha posat informació sensible als prompts. La IA local amb Ollama i un model com `llama3.1` o `mistral` sí que seria totalment privada — i s'explica al M4.

Què passa quan trobis un error

Si trobes alguna cosa que no funciona, és un **bug de la IA**. La millor manera de corregir-lo és:

1. Documentar exactament què has provat i què ha fallat.
2. Enviar la correcció a l'autor (Bernat Mora).
3. Ell decideix si val la pena actualitzar el llibre.

Aquest és un llibre viu. No està gravat en pedra. Cada nova versió és una oportunitat per millorar.

Agraïments

A Bernat Mora, per la paciència infinita d'anar validant cada capítol a la seva Raspberry, per la cura de revisar cada secció, i per la voluntat de mantenir el llibre honest sobre els seus orígens.

A la comunitat de programari lliure, per totes les eines que fan servir tant aquest llibre com el BernatLab: Debian, Docker, Tailscale, MQTT, Grafana, InfluxDB, Node-RED, Ollama, i tantes altres.

A tu, lector o lectora, per arribar fins aquí. Si has llegit el pròleg, és perquè t'interessa no només el què, sinó també el com. Això és bon senyal.

Comencem.

Capítol 58 — Preparació: què necessites tenir abans de començar

"El 80% dels problemes d'una instal·lació es decideixen abans de tocar cap botó. Si prepares bé la llista de la compra i el pla, el dia D flueix."

58.1 A qui va dirigit aquest mòdul

Aquest mòdul és diferent dels anteriors. Allà on el **M1** explicava conceptes i el **M5** assegurava el sistema, **aquest mòdul t'acompanya pas a pas** a muntar el teu propi BernatLab des de zero.

Si ja tens la Raspberry en marxa, pots saltar-te les parts que ja has fet. Però si vols oferir el BernatLab a un amic o familiar — o si vols refer-lo tot de nou per entendre'l bé — segueix l'ordre dels capítols.

Al final d'aquest mòdul tindràs:

- Una Raspberry Pi 4 preparada i segura.
- Un servidor de monitoratge (Uptime Kuma, Prometheus, Grafana).
- Una cadena de dades funcional (MQTT → InfluxDB → Grafana).
- Un node LoRa enviant dades reals des del camp.
- Un bot de Telegram que t'avisa quan passa alguna cosa.
- Un runbook per si alguna cosa es trenca.
- Un DRP que has provat, no només escrit.

58.2 Què necessites: la llista de la compra

Això és el que jo tenia el dia que vaig començar. No és la llista mínima universal, és la llista que **a mi m'ha funcionat** i que et recomano.

Hardware essencial

Component	Quantitat	Preu aprox.	Per què
Raspberry Pi 4 Model B (4 GB RAM)	1	55 €	El cor del BernatLab. 4 GB és el mínim; 8 GB és el recomanat.
MicroSD A2 de 64 GB	1	15 €	Disc del sistema. **A2** vol dir bona velocitat d'escriptura.
SSD USB 3.0 de 500 GB	1	50 €	On van les dades. La microSD falla; la SSD és més segura.
Carcassa amb dissipador i ventilador	1	15 €	La RPi 4 s'escalfa; sense refrigeració el rendiment és baix.
Font d'alimentació USB-C oficial (5V/3A)	1	12 €	**L'oficial** . Les fonts barates donen problemes.
Cable Ethernet Cat 6 (2 m)	1	5 €	Millor que Wi-Fi: més estable, més ràpid, més segur.

Hardware recomanable

Component	Quantitat	Preu aprox.	Per què
SSD NVMe amb adaptador USB 3.0		80 €	Més ràpid que SSD SATA. Si vols anar sob
Cables Dupont i protoboard	1 lot	10 €	Per connectar sensors al GPIO.
Sensor BME280 (I2C)	1	5 €	Temperatura, humitat, pressió. Per a un no
Sensor d'humitat del sòl capacitiu		3 €	Per a un node exterior.
Mòdul SX1262 868 MHz + antena		15 €	Per al node LoRa.

Eines i accessoris

- Un **ordinador** amb el qual configurar la RPi (Windows, Mac o Linux). El teu dia a dia.
- Un **lector de microSD** USB, per flashear la imatge.
- Un **monitor amb HDMI + teclat USB**, per si mai necessites accés físic. No cal comprar-ne un de nou: qualsevol monitor vell serveix.
- Un **cable USB-C** llarg (1-2 m) per si la RPi queda lluny del router.
- **Etiquetes adhesives** per marcar cables, ports, contenidors.
- Una **caixa o calaix** per emmagatzemar la còpia de seguretat del sistema.

Software

- **Raspberry Pi Imager** (gratuït): <https://www.raspberrypi.com/software/>
- **PuTTY** (Windows) o **Terminal** (Mac/Linux): per accedir per SSH.
- **Restic**: `sudo apt install restic` un cop tinguis la RPi en marxa.

Comptes que necessitaràs crear

- **Tailscale** (gratuït per a ús personal): <https://tailscale.com>
- **GitHub** (gratuït): per al teu repo i per descarregar eines.
- **The Things Network** (gratuït): per als nodes LoRaWAN, si esculls aquesta via.
- **Backblaze B2** (gratuït fins a 10 GB): per a les còpies al núvol.
- **Telegram** (gratuït): per al bot d'alertes.

58.3 Quin cost total esperar

Si compres tot de zero, el cost aproximat és:

- Hardware essencial: 152 €
- Hardware recomanable: 30 €
- Software i eines: 0 € (tot gratuït o ja el tens)

- Comptes al núvol: 0-3 €/mes (restic, B2, etc.)

Total: ~180 € d'entrada, 3 €/mes de manteniment.

És un cost raonable per a un servidor 24/7 que t'allotjarà dades, automatitzacions, alertes, IA, i qualsevol cosa que hi vulguis connectar.

58.4 Quin temps invertir

Depèn de com ho facis:

Modalitat	Temps	Comentari
Amb algú que t'ajuda	6-8 hores	Si tens un amic que ja ho ha fet.
Sol, seguint el llibre	12-15 hores	Al teu ritme, amb temps per assimilar.
Sol, sense pauses	8-10 hores	Si tens pressa, en un cap de setmana.

Jo recomano la **modalitat sol, seguint el llibre**, repartida en 3-4 sessions d'un cap de setmana. Tens temps per llegir, entendre i adaptar.

58.5 Quin coneixement necessites

Aquest llibre no assumeix que siguis expert, però tampoc comença des de zero. Hauries de saber:

- **Fer servir la línia de comandes bàsica:** `cd`, `ls`, `cp`, `mv`. Si no, llegeix el **Cap 3** (Linux per administrar un servidor) abans.
- **Llegir documentació tècnica:** la majoria d'eines tenen documentació anglesa. Et pots apanyar amb el català, però l'anglès t'ajudarà.
- **Editar fitxers de text:** nano o vim. El **Cap 3** t'ensenya nano.
- **Tenir paciència:** les coses fallen. De vegades, l'única solució és tornar-ho a provar.

Si tot això et sona però no ho has fet mai, **perfecte**. Aprendràs fent.

58.6 Quin espai físic

La Raspberry és petita, però pensa on la posaràs:

Criteris

- **Ventilació:** no la tanquis en un armari hermètic. Li cal aire.
- **Protecció:** pols, esquitxos, animals (gats, ratolins, formigues).
- **Xarxa a prop:** com més a prop del router, millor. Cable Ethernet millor que Wi-Fi.
- **Endoll:** a prop d'un endoll o regleta amb protecció contra sobretensions.
- **Protecció contra talls de llum:** si tens un **SAI** (Sistema d'Alimentació Ininterrompuda, com els que es venen per a PCs), millor. La RPi consumeix poc i un SAI petit la pot mantenir una estona.

Possibles ubicacions

- A la sala d'estar, al costat del router, en una caixa ventilada.
- Al despatx, en una prestatgeria.
- A l'armari de les comunicacions (on tens el router, l'ONT de la fibra, etc.).
- A l'hort mateix, si tens electricitat i cobertura Wi-Fi allà (amb protecció IP65).

Jo la tinc a la sala d'estar, en una caixa de fusta amb ventilació. Fa 3 anys que hi és i no ha fallat mai.

58.7 Quines decisions prendre abans de començar

Algunes decisions que has de prendre **abans** de tocar res:

1. Nom del servidor

Li posarem `hortosona` al llarg d'aquest llibre, però tu pots triar el que vulguis. Consells:

- Tot minúscules, sense espais ni caràcters estranys.
- Curt (8-12 caràcters).
- Que soni bé quan hi parlis ("el servidor hortosona està caigut").

2. Subdomini (si en tens un)

Si tens un domini propi (per exemple, `bernat.cat`), pots fer servir subdominis:

- `hortosona.bernat.cat` per accedir al servidor.
- `grafana.bernat.cat` per al panell de Grafana.
- `mosquitto.bernat.cat` per al broker MQTT.

Si no tens domini, **no passa res**. Tailscale et dona noms automàticament (`hortosona.tailnet.ts.net`) que funcionen igual.

3. Quins serveis vull tenir

Al capítol 1 del llibre tens la llista de serveis possibles. Pensa quins necessites realment:

- **Hort Osona:** MQTT, InfluxDB, Grafana, Node-RED.
- **IA local:** Ollama al Mac, integració amb el BernatLab.
- **Còpies al núvol:** Backblaze B2, restic.
- **Monitoratge:** Uptime Kuma, Prometheus, Alertmanager.
- **Web pública:** Nginx Proxy Manager, Cloudflare Tunnel.
- **Altres:** File Browser, Nextcloud, Immich (fotos), etc.

No activis serveis que no necessitis. **Cada servei és feina de manteniment.**

4. Quin hardware extra

Si vols connectar sensors reals, decideix ara quins:

- Sensor interior (BME280): 5 €.
- Sensor exterior (humitat del sòl): 3 €.
- Estació meteorològica completa: 200-500 €.
- Node LoRa: 30-50 € (mòdul + ESP32 + caixa + antena).
- Càmera: 30-50 €.

Comença amb el mínim. Pots afegir més endavant.

58.8 Quin pla de treball

Aquest mòdul està pensat per anar pas a pas. La seqüència recomanada és:

1. **Cap 58** (aquest): Preparació. Estàs aquí.
2. **Cap 59**: Primer contacte amb la Raspberry.
3. **Cap 60**: Sistema base segur.
4. **Cap 61**: Docker i Portainer.
5. **Cap 62**: Uptime Kuma.
6. **Cap 63**: Stack de dades: MQTT, InfluxDB, Grafana.
7. **Cap 64**: Node-RED: automatitzacions reals.
8. **Cap 65**: Node LoRa al camp.
9. **Cap 66**: Bot de Telegram.
10. **Cap 67**: Prometheus i alertes avançades.
11. **Cap 68**: Runbooks: quan falla alguna cosa.
12. **Cap 69**: DRP: el dia que es crema tot.

Cada capítol té una **durada estimada** al principi, i un **checklist** al final. Si la teva durada és molt més llarga, **atura't i pregunta**. Millor trigar més que no pas fer-ho malament.

58.9 Què tenir a mà durant la instal·lació

Quan et posis a treballar, tingues a mà:

- Aquest llibre (obert a la pantalla, o imprès si ho prefereixes).
- Un quadern o fitxer de notes per anotar:
- Comandes que has executat.
- Errors que has vist.
- Decisions que has pres.
- Contrasenyes (que després mouràs a un gestor).
- Un **termòmetre de ambient** (no és broma — la temperatura de la RPi afecta el rendiment).
- Un **tester de xarxa** o simplement poder fer **ping** des d'una altra màquina.

58.10 Com documentar la teva instal·lació

Al repo del BernatLab, ja tens un `homelab/` buit. Crea-hi:

```
homelab/
■■■ README.md           # Descripció general
■■■ setup-log.md         # El diari de la teva instal·lació
■■■ decisions.md        # Les decisions que has pres i per què
■■■ compose/            # Els teus fitxers docker-compose
```

El **setup-log.md** és especialment valuós. Anota cada pas amb:

- Data i hora.
- Comanda executada.
- Sortida esperada.
- Sortida real (especialment si falla).
- Solució aplicada.

D'aquí un any, quan oblidis com vas fer una cosa, obriràs aquest fitxer i recordaràs.

58.11 Errors habituals en la preparació

Error 1: comprar la primera RPi que trobes.

Assegura't que és **Raspberry Pi 4 Model B** (o Pi 5 si el pressupost ho permet). No confondre amb RPi 3 (més antiga) o RPi Zero (molt menys potent).

Error 2: comprar una microSD qualsevol.

Les **SanDisk Extreme** o **Samsung EVO** amb classe **A2** són les recomanades. Una microSD lenta farà que el sistema vagi lent.

Error 3: estalviar en la font d'alimentació.

La RPi 4 necessita 5V/3A. Una font de 5V/2A (la del mòbil) no és prou. Sortirà "low voltage warning" i el rendiment baixarà.

Error 4: no tenir un pla B.

Si compres la RPi, la microSD, però no el cable Ethernet, no podràs configurar res. Mira la llista sencera abans de comprar.

Error 5: començar divendres a la nit.

Si comences un projecte llarg un divendres a la nit, acabaràs amb un sistema mig muntat i frustració. Comença en un moment amb temps de sobres.

58.12 Com organitzar les còpies de seguretat

Des del primer moment, pensa en còpies:

- Una **microSD extra** amb la imatge flashejada (per si falla la principal).
- Un **disc USB** per fer la primera còpia un cop tinguis el sistema en marxa.
- Un compte de **Backblaze B2** creat ja, per tenir la còpia al núvol configurada des del primer dia.

La regla 3-2-1 (3 còpies, 2 suports, 1 fora de casa) és important **des del primer moment**, no pas quan ja tinguis dades valuoses.

58.13 El "moment de la veritat": quan ho engegues tot

El dia que connectis la RPi per primera vegada, tindràs una sensació d'**emoció barrejada amb por**. És normal. Si alguna cosa no va, recorda:

- El 80% dels problemes tenen solució buscant el missatge d'error exacte.
- El 20% restant es resol apagant i tornant a encendre (no és broma — funciona més del que voldríem).
- Si no trobes la solució, deixa-ho, dorm, i torna-hi l'endemà amb ulls frescos.

58.14 Resum

Aquest capítol és la "llista de la compra" del BernatLab. Hem vist:

- Quin hardware cal (essencial i recomanable).
- Quin cost esperar (~180 € d'entrada, 3 €/mes).
- Quin temps invertir (12-15 hores).
- Quin coneixement cal tenir.
- Quin espai físic.
- Quines decisions prendre.
- Quin pla seguir.

Si tens tot això clar, ja pots passar al **Cap 59**: Primer contacte amb la Raspberry.

58.15 Exercicis pràctics

1. Fes la teva pròpia llista de la compra i compara-la amb la meva.
2. Calcula el cost total del teu projecte.
3. Decideix el nom del servidor.
4. Decideix on el posaràs.
5. Crea el compte de Tailscale.
6. Crea el compte de Backblaze B2.
7. Crea el compte de GitHub (si no el tens).
8. Comença el quadern de notes.

Capítol 59 — Primer contacte amb la Raspberry Pi

"Aquest és el moment de la veritat. Connectes la RPi, prem el botó, i tens un servidor. Si tot va bé, deu segons. Si no, mitja hora. Benvingut al món real."

59.1 Què aprendràs

Al final d'aquest capítol tindràs:

- La Raspberry Pi amb la microSD flashejada i arrancada.
- Connexió a la teva xarxa local (via Ethernet).
- Accés per SSH des del teu ordinador.
- L'adreça IP de la Raspberry per connectar-hi.
- Les primeres comandes executades.

Aquest capítol és **pràctic al 100%**. Llegir-lo sense fer-ho no serveix de gaire. Té la RPi a mà quan comencis.

59.2 Durada estimada

- Amb experiència: 15-20 min.
- Primer cop: 45-60 min (comptant possibles contratemps).
- Si mai has flashejat una microSD: +30 min.

59.3 Material a mà

Assegura't que tens:

- La **Raspberry Pi 4** amb la carcassa posada.
- La **microSD** amb el seu adaptador USB.
- La **font d'alimentació oficial** USB-C.
- El **cable Ethernet** connectat a la RPi i al router.
- Un **ordinador** amb el qual accedir per SSH.
- Un **monitor amb HDMI** i teclat USB (opcional, per si fallen coses).

59.4 Pas 1: descarregar i instal·lar Raspberry Pi Imager

Al teu **ordinador** (no a la RPi encara), descarrega **Raspberry Pi Imager** des de:

- <https://www.raspberrypi.com/software/>

Està disponible per a Windows, Mac i Linux. Instal·la'l com qualsevol programa.

59.5 Pas 2: triar el sistema operatiu

Obre Raspberry Pi Imager. A la primera pantalla veuràs tres menús:

1. **Dispositiu**: tria "Raspberry Pi 4".
2. **Sistema operatiu**: tria "Raspberry Pi OS (other)" → "Raspberry Pi OS Lite (64-bit)".
3. **Emmagatzematge**: tria la teva microSD.

Per què **Lite** i no l'escriptori complet?

- **Lite** no té entorn gràfic, és només terminal. Ocupa menys espai i menys RAM.
- Per a un servidor 24/7, l'escriptori és innecessari.
- Si vols, sempre pots afegir-lo més tard.

59.6 Pas 3: configuració personalitzada

Abans de flashejar, Imager et permet configurar quatre coses útils. **Això t'estalviarà 30 minuts** després.

Fes clic a "**Edita la configuració**" (o "Edit Settings" en anglès). S'obrirà una finestra amb quatre pestanyes:

Pestanya "General"

- **Hostname**: escriu `hortosona` (o el nom que hagis triat). Això serà el nom de la RPi a la xarxa.
- **Nom d'usuari**: jo faig servir `bernat`, tu pots fer servir el que vulguis. Però ****no facis servir `pi`****: és el nom per defecte i tots els bots d'Internet el coneixen.
- **Contrasenya**: una de forta, com les del **Cap 46** (2FA i secrets). Mínim 16 caràcters. Guarda-la al teu gestor de contrasenyes.
- **Wi-Fi**: deixa-ho buit. Farem servir Ethernet.
- **Zona horària**: posa-hi la teva (per a Catalunya, `Europe/Madrid` serveix).
- **Layout del teclat**: si el teu és espanyol, tria `es`.

Pestanya "Serveis"

- Activa "**Enable SSH**".
- Triar "**Use password authentication**" és acceptable per començar. Més endavant (al capítol 60) canviarem a clau pública.
- Si vols, pots activar "Serial console" per si mai necessites accedir via UART.

Pestanya "Opcions"

- Pots deixar-ho tot per defecte.

Fes clic a "**Desar**" i després a "**Escriure**" (o "Write").

59.7 Pas 4: flashejar la microSD

Imager començarà a escriure. Això triga entre 1 i 5 minuts, depenent de la microSD. **Paciència:** no treguis la targeta a mitja escriptura.

Quan acabi, Imager et preguntarà si vols extreure la targeta. Diu-li que sí, treu-la, i fica-la a la RPi.

59.8 Pas 5: primer arrencada de la Raspberry

Amb la microSD ja inserida:

1. Connecta el **cable Ethernet** de la RPi al router.
2. Connecta la **font d'alimentació** USB-C.
3. La RPi arrencarà automàticament. No té botó d'encesa (o si en té, no cal prémer-lo).

Esperem que arrenqui. La **LED vermella** ha de quedar fixa, i la **LED verda** ha de parpellejar durant 10-30 segons mentre llegeix la microSD.

Quan la verda deixi de parpellejar, el sistema ha acabat d'arrencar.

59.9 Pas 6: trobar l'adreça IP

Ara ve la part interessant: descobrir quina IP ha agafat la RPi al router. Hi ha diverses maneres:

Opció A: mirar al router

La majoria de routers tenen una pàgina d'administració on pots veure els dispositius connectats. Normalment és `http://192.168.1.1` o `http://192.168.0.1`. Busca a la llista un dispositiu amb nom `hortosona` i apunta la seva IP.

Opció B: fer ping al hostname

Des del teu ordinador:

```
ping hortosona
```

Si funciona, ja tens la IP. Si no, el teu ordinador no resol noms per mDNS (cosa que passa al Windows).

Opció C: eines específiques

- **Windows:** pots instal·lar **Advanced IP Scanner** (gratuït) o fer servir `arp -a` per veure dispositius de la xarxa.
- **Mac/Linux:** `arp -a` o `nmap -sn 192.168.1.0/24` (si tens `nmap` instal·lat).
- **Apps mòbils:** **Fing** (iOS/Android) és excel·lent per trobar dispositius a la xarxa.

Opció D: provar IPs comuns

Si tens un router amb DHCP normal, la RPi estarà a alguna IP com `192.168.1.42` o `192.168.0.42`. Prova unes quantes.

Jo normalment uso la opció A (mirar al router) perquè és la més fiable.

59.10 Pas 7: accedir per SSH

Un cop tens la IP, obre un terminal al teu ordinador:

Windows (PowerShell o CMD):

```
ssh bernat@hortosona
```

Si tens Windows 10/11 modern, `ssh` ja està disponible. Si no, instal·la OpenSSH o usa PuTTY.

Mac/Linux:

```
ssh bernat@hortosona
```

La primera vegada, SSH et preguntarà si confies en aquest host. Escriu `yes`.

Després et demanarà la contrasenya (la que vas posar a la Imager). Escriu-la (no es veurà mentre l'escrius, és normal).

Si tot va bé, veuràs alguna cosa com:

```
Linux hortosona 6.6.51+rpt-rpi-v8 #1 SMP PREEMPT Debian 1:6.6.51-1+rpt3 (2024-10-08) aarch64
```

```
The programs included with the Debian GNU/Linux system are free software;
```

```
...
```

```
bernat@hortosona:~ $
```

Felicitats! Acabes d'entrar al teu primer servidor.

59.11 Pas 8: les primeres comandes

Un cop dins, executa aquestes comandes per entendre on ets:

```
# Qui sóc jo
whoami
```

```
# On sóc
pwd
```

```
# Quina distro tinc
cat /etc/os-release
```

```
# Quin hardware tinc
uname -a
lscpu
free -h
df -h
```

Hauries de veure:

- `whoami` retorna `bernat` (o el teu nom d'usuari).
- `pwd` retorna `/home/bernat`.
- La distro és **Debian GNU/Linux 13 (trixie)**.
- L'arquitectura és **aarch64** (ARM 64 bits).
- La CPU és una **ARM Cortex-A72** amb 4 nuclis.
- La RAM és la que hakis comprat (4 GB o 8 GB).
- L'espai lliure a la microSD.

59.12 Pas 9: configurar la xarxa correctament

Si la IP t'ha canviat a cada reinici, és perquè el DHCP del router t'està donant IPs dinàmiques. Per evitar-ho, podem fer dues coses:

Opció A: IP estàtica al router

Mira al router quin rang de DHCP té, i assigna una IP fixa a la MAC de la RPi. Cada router és diferent, però busca "DHCP reservation" o "IP estàtica" al menú.

Això és el que jo faig: la RPi sempre té la mateixa IP (excepte amb Tailscale, on té una IP del tailnet).

Opció B: IP estàtica a la RPi

Edita la configuració de xarxa:

```
sudo nano /etc/dhcpd.conf
```

Afegeix al final:

```
interface eth0
static ip_address=192.168.1.100/24
static routers=192.168.1.1
static domain_name_servers=192.168.1.1 1.1.1.1
```

(Canvia les IPs per les de la teva xarxa.)

Guarda amb **Ctrl+O**, surt amb **Ctrl+X**, i reinicia:

```
sudo systemctl restart dhcpd
```

Per què és important tenir una IP fixa

Sense IP fixa, cada vegada que la RPi es reiniciï, el router li pot donar una IP diferent. Això és molt molest per:

- Scripts d'automatització.
- Configurar altres dispositius per apuntar a aquesta IP.
- Recordar on és el servidor.

Tailscale ho soluciona (et dona una IP fixa del tailnet), però tenir la IP local fixa també ajuda.

59.13 Pas 10: primeres actualitzacions

Un cop tens accés, actualitza el sistema:

```
sudo apt update
sudo apt upgrade -y
```

Això pot trigar una estona. Quan acabi, reinicia:

```
sudo reboot
```

Després d'un parell de minuts, torna a entrar per SSH:

```
ssh bernat@hortosona
```

59.14 Pas 11: fer la primera còpia de seguretat de la microSD

Ara que el sistema està net i actualitzat, **abans de fer res més**, fes una còpia de la microSD. Si la RPi falla demà, podràs restaurar en 5 minuts.

Apaga la RPi:

```
sudo shutdown -h now
```

Després d'un minut, la LED verda parpalleja 10 vegades i s'apaga. Treu la microSD.

Al teu ordinador, amb un lector de microSD:

Windows: pots usar **Win32 Disk Imager** o **balenaEtcher** (sí, el mateix que per flashear).

Mac:

```
diskutil list
# Troba la microSD (ex: /dev/disk2)
diskutil unmountDisk /dev/disk2
sudo dd if=/dev/rdisk2 of=~/backup-raspberry-2026-07-09.img bs=4m status=progress
```

Linux:

```
lsblk
# Troba la microSD (ex: /dev/sdb)
sudo dd if=/dev/sdb of=~/backup-raspberry-2026-07-09.img bs=4M status=progress
```

Comprimeix la imatge:

```
gzip ~/backup-raspberry-2026-07-09.img
```

Guarda aquest fitxer en un lloc segur. Si tens una còpia al núvol (Backblaze B2, per exemple), millor.

59.15 Què has après

Ara ja tens:

- Una Raspberry Pi amb sistema operatiu.
- Connexió SSH des del teu ordinador.
- Una adreça IP fixa a la xarxa.
- El sistema actualitzat.
- Una còpia de seguretat de la microSD.

Al **Cap 60** farem el següent:

- Canviar la contrasenya per una clau SSH.
- Instal·lar Tailscale per accedir des de qualsevol lloc.
- Configurar l'usuari correctament.
- Aplicar les primeres mesures de seguretat.

59.16 Errors habituals

Error 1: la LED vermella parpalleja.

Vol dir que la font d'alimentació no és prou potent. Canvia-la per l'oficial.

Error 2: la LED verda no parpelleja mai.

La microSD no està ben flashejada, o no està ben insertada. Torna a flashear.

Error 3: "Permission denied" en fer SSH.

Contrasenya incorrecta, o usuari mal escrit. Recorda que Linux distingeix majúscules.

Error 4: "Connection refused".

La RPi no està en marxa, o SSH no està activat. Connecta un monitor per veure què passa.

Error 5: "Host key verification failed".

Estàs intentant accedir a una RPi que ja tens enregistrada amb una clau diferent. Solució:

`ssh-keygen -R hortosona` al teu ordinador.

Error 6: el router no mostra la RPi.

Comprova que el cable Ethernet està ben connectat a les dues bandes. Mira si el LED de xarxa de la RPi s'il·lumina (verd o groc al port Ethernet).

Error 7: l'IP canvia a cada reinici.

Configura una IP estàtica, com hem explicat al pas 12.

59.17 Resum

Aquest capítol és el moment de la veritat: la RPi en marxa, accessible per SSH, amb una còpia de seguretat. Hem vist:

- Com flashear la microSD amb Raspberry Pi Imager.
- Com triar el sistema operatiu correcte (Lite, 64-bit).
- Com configurar coses abans de flashear (estalvia temps).
- Com trobar la IP de la RPi.
- Com accedir per SSH.
- Com actualitzar el sistema.
- Com fer la primera còpia de seguretat.

Al **Cap 60** endurem la seguretat: clau SSH, Tailscale, i primers ajustos.

59.18 Exercicis pràctics

1. Descarrega Raspberry Pi Imager.
2. Flasheja la microSD amb Raspberry Pi OS Lite 64-bit.
3. Configura el hostname, usuari i contrasenya a Imager.
4. Activa SSH amb autenticació per contrasenya.
5. Insereix la microSD a la RPi i engega-la.
6. Troba la IP de la RPi.

7. Accedeix per SSH.
8. Executa les comandes del pas 11 per entendre el sistema.
9. Configura una IP estàtica.
10. Actualitza el sistema.
11. Fes una còpia de seguretat de la microSD.
12. Documenta-ho tot al `homelab/setup-log.md`.

Capítol 60 — Sistema base segur

*“La diferència entre un servidor vulnerable i un de segur és, sovint, una hora de feina. Aquest capítol és aquesta hora.”**

60.1 Què aprendràs

Al final d'aquest capítol tindràs:

- Autenticació per **clau SSH** (sense contrasenya).
- **Contrasenya desactivada** per SSH (només clau).
- **Tailscale** instal·lat i configurat.
- Accés a la RPi des de qualsevol lloc del món via Tailscale.
- **2FA** per a SSH (opcional, però recomanat).
- Les primeres mesures de **hardening** aplicades.

Aquest és el capítol on el teu servidor passa de "joguina" a "infraestructura". Tot el que hi ha aquí és **defensa en profunditat** (recorda el **Cap 43**, filosofia de seguretat).

60.2 Durada estimada

- Amb experiència: 30-45 min.
- Primer cop: 1-1.5 hores (comptant la generació de claus).

60.3 Abans de començar

Assegura't que:

- Estàs connectat a la RPi per SSH.
- Tens accés a un altre terminal (el del teu ordinador, per generar les claus).
- Tens **dos dispositius preparats**: la RPi i el teu ordinador.

Si tens problemes durant el capítol, recorda que **no has de tancar la sessió SSH actual** fins que tot funcioni. Si la configures malament, et pots quedar fora.

60.4 Pas 1: generar una clau SSH forta

Al **teu ordinador** (no a la RPi), obre un terminal i genera una clau SSH:

```
ssh-keygen -t ed25519 -C "bernat@bernat-mbp" -f ~/.ssh/bernatlab
```

Explicació:

- **-t ed25519**: tipus de clau. Ed25519 és moderna, curta i segura.
- **-C "..."**: comentari per identificar-la (el teu correu o usuari).

- `~/.ssh/bernatlab`: on guardar-la. **No usis el nom per defecte** (`id_ed25519`), perquè si més endavant tens altres claus per a altres servidors, no es barregen.

Et demanarà una **passphrase**. **Sempre posa'n una**. Això xifra la clau privada al teu disc: si algú te la roba, no pot fer-la servir.

Recorda la passphrase al teu gestor de contrasenyes.

Això genera dos fitxers:

- `~/.ssh/bernatlab` (clau privada — **no la comparteixis mai**).
- `~/.ssh/bernatlab.pub` (clau pública — la pots penjar a la RPi).

60.5 Pas 2: afegir la clau pública a la Raspberry

Des del teu **ordinador**, copia la clau pública a la RPi:

```
ssh-copy-id -i ~/.ssh/bernatlab.pub bernat@hortosona
```

Et demanarà la contrasenya (l'última vegada que l'hauràs de posar). Quan acabi, la clau pública estarà a `~/.ssh/authorized_keys` de la RPi.

Alternativa manual (si `ssh-copy-id` no funciona):

```
cat ~/.ssh/bernatlab.pub | ssh bernat@hortosona "mkdir -p ~/.ssh && chmod 700 ~/.ssh && cat >> ~/.ssh/authorized_keys"
```

60.6 Pas 3: provar la connexió amb clau

Ara intenta entrar amb la clau:

```
ssh -i ~/.ssh/bernatlab bernat@hortosona
```

Si et demana la passphrase de la clau (no la contrasenya!), és que tot funciona. Si entra directament sense demanar res, tens `ssh-agent` actiu i la clau ja està desxifrada.

Per evitar haver d'escriure la passphrase cada vegada, afegeix la clau a l'agent:

Mac:

```
ssh-add --apple-use-keychain ~/.ssh/bernatlab
```

Linux:

```
ssh-add ~/.ssh/bernatlab
```

Ara ja no hauries de necessitar posar la passphrase cada vegada.

60.7 Pas 4: simplificar la configuració SSH al teu ordinador

Al teu **ordinador**, crea o edita `~/.ssh/config`:

Mac/Linux:

```
nano ~/.ssh/config
```

Windows (PowerShell): `notepad $HOME\.ssh\config`

Afegeix:

```
Host hortosona
```

```
HostName 192.168.1.100
User bernat
IdentityFile ~/.ssh/bernatlab
IdentitiesOnly yes
```

Així, només amb `ssh hortosona`, el teu ordinador sap que ha de connectar a la IP correcta amb la clau correcta.

Si tens Tailscale configurat (pas següent), pots canviar el `HostName` per la IP del tailnet:

```
Host hortosona
  HostName 100.115.134.76
  User bernat
  IdentityFile ~/.ssh/bernatlab
  IdentitiesOnly yes
```

60.8 Pas 5: instal·lar Tailscale

Tailscale et permet accedir a la teva RPi des de qualsevol lloc del món, sense obrir ports al router. És una VPN moderna basada en WireGuard.

A la **RPi**:

```
curl -fsSL https://tailscale.com/install.sh | sh
```

Després:

```
sudo tailscale up
```

Això et donarà una URL. Obre-la al navegador, autentica't amb el compte que has creat al **Cap 58**, i la RPi quedarà afegida al teu tailnet.

Verifica:

```
tailscale status
```

Hauries de veure la RPi amb una IP del tipus `100.x.x.x`. Aquesta és la IP Tailscale.

60.9 Pas 6: provar l'accés per Tailscale

Ara, **des del teu ordinador** (o des del mòbil!), intenta accedir per la IP Tailscale:

```
ssh bernat@100.115.134.76
```

(Canvia la IP per la que t'hagi tocat.)

Si funciona, ja tens accés a la RPi des de qualsevol lloc amb Tailscale instal·lat. Sense obrir cap port al router, sense IP pública, sense res.

Si tens un amic amb qui vulguis compartir accés (per exemple, per ajudar-te amb el servidor), pots afegir-lo al teu tailnet des de la consola de Tailscale: <https://login.tailscale.com/admin>.

60.10 Pas 7: desactivar l'accés per contrasenya

Un cop la clau SSH funciona, **desactiva l'autenticació per contrasenya**. Això evita atacs de força bruta.

A la **RPi**:

```
sudo nano /etc/ssh/sshd_config
```

Cerca o afegeix aquestes línies (assegurant-te que no estan comentades amb #):

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
PermitEmptyPasswords no
```

Per trobar-les ràpid, pots fer:

```
sudo grep -E "^#?(PermitRootLogin|PasswordAuthentication|PubkeyAuthentication|PermitEmptyPasswords)" /etc
```

Si una línia està comentada (`#PasswordAuthentication yes`), lleva-li el `#` i canvia el valor a `no`.

Desa i reinicia SSH:

```
sudo systemctl restart sshd
```

Important: no tanquis la sessió SSH actual! Prova primer des d'un altre terminal:

```
ssh bernat@hortosona
```

Si funciona, llavors pots tancar la sessió original.

60.11 Pas 8: instal·lar 2FA a SSH (opcional però recomanat)

Afegeix una capa extra amb 2FA. Així, encara que robin la teva clau SSH, no podran entrar.

A la **RPi**:

```
sudo apt install libpam-google-authenticator
```

Configura per al teu usuari:

```
google-authenticator
```

Et mostrarà un codi QR. Escaneja'l amb Google Authenticator, Authy o 1Password al teu mòbil.

Guarda els **codis de recuperació** en un lloc segur (Bitwarden, paper en una caixa forta). Si perds el mòbil, aquests codis et deixaran entrar.

Edita la configuració de PAM:

```
sudo nano /etc/pam.d/sshd
```

Afegeix al final:

```
auth required pam_google_authenticator.so
```

Edita la configuració de SSH:

```
sudo nano /etc/ssh/sshd_config
```

Canvia o afegeix:

```
ChallengeResponseAuthentication yes
AuthenticationMethods publickey,keyboard-interactive
```

Reinicia SSH:

```
sudo systemctl restart sshd
```

Prova des d'un altre terminal:

```
ssh bernat@hortosona
```

Ara hauries de veure:

1. Contrasenya de la clau (si tens passphrase).
2. Codi de Google Authenticator.

Si tot funciona, ja tens SSH amb clau + 2FA. **Molt segur.**

60.12 Pas 9: configurar actualitzacions automàtiques

Hem parlat d'això al **Cap 48** (hardening). Apliquem-ho ara:

```
sudo apt install unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades
```

Tria "Yes" a la pregunta.

Edita la configuració:

```
sudo nano /etc/apt/apt.conf.d/50unattended-upgrades
```

Assegura't que tens aquestes línies (algunes poden estar comentades):

```
Unattended-Upgrade::Allowed-Origins {
    "Debian bookworm-security";
    "Debian bookworm-updates";
};
Unattended-Upgrade::AutoFixInterruptedDpkg "true";
Unattended-Upgrade::Remove-Unused-Kernel-Packages "true";
Unattended-Upgrade::Remove-Unused-Dependencies "true";
Unattended-Upgrade::Automatic-Reboot "false";
```

(Ajusta **bookworm** per la teva versió; a Debian 13 és **trixie**.)

Prova:

```
sudo unattended-upgrades --dry-run
```

Hauries de veure què passaria si s'executés ara.

60.13 Pas 10: configurar el tallafocs (UFW)

Hem parlat d'UFW al **Cap 47** (fail2ban i tallafocs). Apliquem una política restrictiva:

```
sudo apt install ufw
```

Política per defecte: denega tot, permet el necessari:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
```

Permet SSH:

```
# Si tens 2FA, permet només per Tailscale
sudo ufw allow in on tailscale0 to any port 22

# Si vols SSH per la xarxa local també
sudo ufw allow from 192.168.1.0/24 to any port 22
```

Activa:

```
sudo ufw enable
sudo ufw status verbose
```

Important: comprova que pots seguir connectant per SSH abans de fer res més! Si UFW et talla l'accés, hauràs d'anar amb un monitor a la RPi.

60.14 Pas 11: instal·lar fail2ban

Recordeu el **Cap 47**: fail2ban bloqueja IPs que fan massa intents fallits. Activem-lo:

```
sudo apt install fail2ban
```

Crea la configuració local:

```
sudo nano /etc/fail2ban/jail.local
```

Afegeix:

```
[DEFAULT]
bantime = 1h
findtime = 10m
maxretry = 3
ignoreip = 127.0.0.1/8 100.64.0.0/10

[sshd]
enabled = true
port = ssh
filter = sshd
logpath = /var/log/auth.log
maxretry = 3
bantime = 24h
```

El rang `100.64.0.0/10` és la xarxa Tailscale — molt important per no auto-bloquejar-te.

Reinicia:

```
sudo systemctl restart fail2ban
```

Verifica:

```
sudo fail2ban-client status sshd
```

60.15 Pas 12: permisos estrictes als fitxers crítics

Aplica els permisos del **Cap 48** (hardening):

```
sudo chmod 600 /etc/shadow
sudo chmod 600 /etc/gshadow
sudo chmod 644 /etc/passwd
sudo chmod 644 /etc/group
sudo chmod 600 /etc/ssh/sshd_config
```

60.16 Pas 13: crear la primera còpia amb restic

Recordeu el **Cap 45** (còpies de seguretat). Ara que el sistema base està net, fem la primera còpia:

```
sudo apt install restic
```

Crea una carpeta per a les còpies (pot ser a una SSD USB, a la microSD, o al núvol — decideix tu):

```
mkdir -p /home/bernat/backups
```

Inicialitza el repositori:

```
restic init --repo /home/bernat/backups
```

Et demanarà una **contrasenya** per al repo. **Guarda-la al gestor de contrasenyes**. Si la perds, les dades són irrecuperables.

Fes la primera còpia:

```
restic --repo /home/bernat/backups backup \
/home/bernat \
/etc \
/var/lib/docker 2>/dev/null || true
```

(El `2>/dev/null || true` evita errors si la carpeta Docker encara no existeix.)

Comprova:

```
restic --repo /home/bernat/backups snapshots
```

Hauries de veure una còpia recent.

60.17 Pas 14: configurar el hostname i el missatge d'avís

Edita `/etc/issue.net` per mostrar un avís quan algú intenti accedir per SSH:

```
sudo nano /etc/issue.net
```

```
=====
ALERTA: Aquest sistema és d'ús privat.
Tots els accessos queden registrats.
L'accés no autoritzat està prohibit i pot ser perseguit legalment.
=====
```

Activa'l a SSH:

```
sudo nano /etc/ssh/sshd_config
```

Assegura't que tens:

```
Banner /etc/issue.net
```

Reinicia SSH:

```
sudo systemctl restart sshd
```

60.18 Pas 15: fer una còpia de la microSD amb el sistema base segur

Ara que el sistema està endureït, **torna a fer una còpia de la microSD**. Aquesta és la còpia "bona", la pots restaurar si tot es trenca.

Apaga:

```
sudo shutdown -h now
```

Fes la còpia igual que al **Cap 59** (pas 14). Guarda-la amb un nom clar, com `bernatlab-base-segur-2026-07-09.img`.

60.19 Què has après

Al final d'aquest capítol tens:

- **SSH amb clau** (sense contrasenya).
- **2FA opcional** a SSH.
- **Tailscale** funcionant (accés remot segur).
- **Tallafores UFW** actiu.

- **fail2ban** actiu.
- **Actualitzacions automàtiques** configurades.
- **Còpia de seguretat** inicial feta.
- **Permisos estrictes** als fitxers crítics.
- **Còpia de la microSD** amb el sistema endureït.

Això és la base sobre la qual construirem la resta del BernatLab. Qualsevol cosa que instal·lem a partir d'ara partirà d'aquesta base sòlida.

60.20 Errors habituals

Error 1: pèrdua d'accés SSH.

Si canvies alguna cosa malament, pots perdre l'accés. **Mai tanquis la sessió actual** fins que no hakis provat des d'un altre terminal.

Error 2: UFW et talla l'accés.

Si t'has equivocat amb la regla, UFW et pot tallar SSH. Solució: connectar un monitor + teclat a la RPi i fer `sudo ufw disable`.

Error 3: 2FA et bloqueja.

Si canvies de mòbil o perds els codis, no podràs entrar. **Guarda sempre els codis de recuperació** en un lloc segur.

Error 4: clau SSH sense passphrase.

Si la clau no té passphrase, qualsevol que accedeixi al teu ordinador pot entrar al servidor. Sempre passphrase.

Error 5: oblidar la contrasenya de restic.

Si oblides la contrasenya, les còpies són irrecuperables. Guarda-la al gestor.

60.21 Resum

Aquest capítol converteix la teva RPi en un servidor **professional**. Hem vist:

- Generació i còpia de claus SSH.
- Tailscale per a accés remot segur.
- Desactivació de l'autenticació per contrasenya.
- 2FA per SSH (opcional però recomanat).
- Tallafocs UFW.
- fail2ban.
- Actualitzacions automàtiques.
- Còpia de seguretat amb restic.
- Còpia de la microSD amb el sistema endureït.

Al **Cap 61** instal·larem **Docker** i **Portainer**, la base de tots els serveis del BernatLab.

60.22 Exercicis pràctics

1. Genera una clau SSH ed25519 amb passphrase.
2. Afegeix-la a la RPi.
3. Configura `~/.ssh/config` per simplificar la connexió.
4. Instal·la Tailscale.
5. Desactiva l'autenticació per contrasenya.
6. Activa 2FA a SSH.
7. Configura actualitzacions automàtiques.
8. Activa UFW amb polítiques restrictives.
9. Instal·la fail2ban.
10. Aplica permisos estrictes.
11. Fes la primera còpia amb restic.
12. Fes una còpia de la microSD.
13. Documenta-ho tot al `homelab/setup-log.md`.

Capítol 61 — Docker i Portainer: la base dels serveis

"Si has d'instal·lar una sola aplicació al teu servidor, fes-ho amb Docker. Si n'has d'instal·lar vint, fes-ho també amb Docker. La diferència és que amb la següent opció, vint aplicacions esdevenen un infern de dependències. Amb Docker, segueixen sent vint aplicacions, però cadascuna en una capsula."

61.1 Què aprendràs

- Què és Docker i per què serveix.
- Com instal·lar Docker i Docker Compose.
- Com instal·lar Portainer.
- Com crear el teu primer contenidor.
- Com organitzar els teus serveis amb `docker-compose`.

61.2 Durada estimada

30-45 minuts.

61.3 Per què Docker

A la Raspberry, si instal·léssim cada aplicació directament (`apt install grafana`, `apt install mosquito...`), tindríem un món de:

- Versions barrejades.
- Conflictes de dependències.
- Dificultat per actualitzar.
- Dificultat per reproduir la instal·lació en una altra màquina.

Docker ho soluciona tot. Cada aplicació va en un **contenidor** que conté:

- El codi.
- Les llibreries.
- La configuració.
- Tot el que necessita per funcionar.

I aquests contenidors **no es barregen entre ells**. Pots tenir 20 contenidors corrent simultàniament, cadascun amb les seves pròpies dependències, sense conflictes.

Si alguna cosa falla, esborres el contenidor i el tornes a crear. Com si fos nova.

61.4 Instal·lar Docker

Docker ja ve preparat per a Debian. A la RPi:

```
# Script oficial d'instal·lació
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
rm get-docker.sh
```

Afegeix el teu usuari al grup `docker` per no haver de fer servir `sudo` cada vegada:

```
sudo usermod -aG docker bernat
```

Tanca la sessió SSH i torna a entrar perquè el canvi tingui efecte:

```
exit
ssh bernat@hortosona
```

Verifica:

```
docker --version
docker compose version
docker run hello-world
```

L'últim comanda descarrega una imatge de prova i l'executa. Si funciona, tens Docker correctament instal·lat.

61.5 Instal·lar Portainer

Portainer és una interfície web per gestionar Docker. Per a un homelab és perfecta: pots veure contenidors, imatges, volums, xarxes, logs — tot des del navegador.

Crea una carpeta per a Portainer:

```
mkdir -p ~/homelab/portainer
```

Crea el contenidor:

```
docker run -d -p 9443:9443 --name portainer \
  --restart=always \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v ~/homelab/portainer/data:/data \
  portainer/portainer-ce:latest
```

Explicació:

- `-d`: en segon pla (detached).
- `-p 9443:9443`: exposa el port 9443 (HTTPS) al host.
- `--name portainer`: li posa nom.
- `--restart=always`: si la RPi es reinicia, Portainer torna a engegar.
- `-v /var/run/docker.sock:/var/run/docker.sock`: li dóna accés a Docker.
- `-v ~/homelab/portainer/data:/data`: persistència de dades.

Obre el navegador a <https://hortosona:9443> (o <https://IP-LOCAL:9443>). La primera vegada et demanarà crear un compte d'administrador. **Fes-ho immediatament** — Portainer sense admin és un forat de seguretat.

A la primera configuració, tria "Get Started" (entorn local). Ja tens Portainer.

61.6 Explorar Portainer

Al panell principal veuràs:

- **Stacks:** grups de contenidors (els teus "serveis").
- **Containers:** contenidors individuals.
- **Images:** imatges Docker descarregades.
- **Volumes:** emmagatzematge persistent.
- **Networks:** xarxes virtuals entre contenidors.
- **Events:** registre d'activitat.

Per ara, explora. Encara no tens gaire cosa per mirar.

61.7 Crear el teu primer contenidor a mà

A la terminal, creem un contenidor de prova: **Nginx**, el servidor web més usat del món.

```
docker run -d -p 8080:80 --name nginx-test nginx
```

Ara:

- Portainer mostra el contenidor `nginx-test` en marxa.
- Si obres <http://hortosona:8080> al navegador, veuràs la pàgina de benvinguda de Nginx.

Per veure els logs del contenidor:

```
docker logs nginx-test
```

Per aturar-lo:

```
docker stop nginx-test
```

Per tornar-lo a engegar:

```
docker start nginx-test
```

Per esborrar-lo:

```
docker rm -f nginx-test
```

61.8 Què és Docker Compose

Quan tens més d'un contenidor, gestionar-los amb `docker run` un per un és tediós. **Docker Compose** permet definir tots els teus serveis en un fitxer YAML.

Exemple: un fitxer `docker-compose.yml` per a Portainer:

```
version: "3.8"
```

```
services:
```

```
  portainer:
```

```
    image: portainer/portainer-ce:latest
```

```
    container_name: portainer
```

```
    restart: unless-stopped
```

```
    ports:
```

```
      - "9443:9443"
```

```
    volumes:
```


- /var/run/docker.sock:/var/run/docker.sock
- ./portainer/data:/data

Aquest fitxer fa exactament el mateix que la comanda `docker run` d'abans, però en format declaratiu.

Per engegar:

```
docker compose up -d
```

Per aturar:

```
docker compose down
```

Per veure els logs:

```
docker compose logs -f
```

61.9 Estructura recomanada per a serveis

A la teva RPi, crea aquesta estructura:

```
~/homelab/
├── README.md           # Notes generals
├── compose/            # Tots els docker-compose
│   ├── portainer.yml
│   ├── uptime-kuma.yml
│   ├── mqtt.yml
│   ├── influxdb.yml
│   ├── grafana.yml
│   └── ...
├── data/               # Volumes persistents
│   ├── portainer/
│   ├── uptime-kuma/
│   ├── mqtt/
│   └── ...
├── secrets/            # Fitxers .env amb secrets
│   ├── mqtt.env
│   ├── influxdb.env
│   └── ...
├── scripts/            # Scripts de manteniment
│   ├── backup.sh
│   ├── update.sh
│   └── healthcheck.sh
```

Aquesta estructura t'acompanyarà durant anys.

61.10 Convertir Portainer a Compose

Mou el Portainer que hem creat a mà a un fitxer Compose:

Crea `~/homelab/compose/portainer.yml`:

```
version: "3.8"

services:
  portainer:
    image: portainer/portainer-ce:latest
    container_name: portainer
    restart: unless-stopped
    ports:
      - "9443:9443"
    volumes:
```

- /var/run/docker.sock:/var/run/docker.sock
- ./data/portainer:/data

Aquest fitxer farà servir `./data/portainer/` dins de la carpeta `compose/`, que és la mateixa ruta que ja estem usant.

Atura i esborra el contenidor antic:

```
docker stop portainer
docker rm portainer
```

Engega'l amb Compose:

```
cd ~/homelab/compose
docker compose -f portainer.yml up -d
```

Verifica que funciona obrint <https://hortosona:9443>. Si tot va bé, ja tens Portainer sota Compose.

61.11 Comandes útils de Docker

```
# Veure contenidors en marxa
docker ps
```

```
# Veure tots els contenidors (inclosos aturats)
docker ps -a
```

```
# Veure imatges descarregades
docker images
```

```
# Veure espai ocupat per Docker
docker system df
```

```
# Netejar recursos no utilitzats
docker system prune -a
# Compte: això esborra imatges, contenidors aturats, xarxes no usades.
# NO esborra volums (per seguretat).
```

```
# Veure logs d'un contenidor
docker logs -f portainer
# -f = follow (en temps real)
```

```
# Entrar dins d'un contenidor
docker exec -it portainer /bin/sh
# Per Nginx Alpine, /bin/sh
# Per Debian/Ubuntu, /bin/bash
```

```
# Estadístiques en temps real
docker stats
```

61.12 Bones pràctiques

1. **No posis secrets al `docker-compose.yml`**. Usa un fitxer `.env` a part.
2. ****Posa `restart: unless-stopped`**** a tots els serveis. Si la RPi es reinicia, tot torna a engegar.
3. **Fes servir volums** per a totes les dades persistents. Mai guardis dades dins del contenidor.
4. **Etiqueta les imatges**: usa tags específics (`nginx:1.25.3`) en lloc de `:latest`.
5. **Limita els recursos** dels contenidors que poden consumir molt.

6. **Monitora** els contenidors (capítol 67).

61.13 Què ve després

Ara tens Docker i Portainer funcionant. Al **Cap 62** afegirem el primer servei de monitoratge: **Uptime Kuma**.

61.14 Resum

Docker + Portainer és la base de tot el que vindrà. Hem vist:

- Què és Docker i per què serveix.
- Com instal·lar Docker i Docker Compose.
- Com instal·lar Portainer i gestionar-lo via web.
- Comandes bàsiques de Docker.
- L'estructura recomanada per als teus serveis.
- Bones pràctiques.

61.15 Exercicis pràctics

1. Instal·la Docker.
2. Afegeix el teu usuari al grup docker.
3. Instal·la Portainer.
4. Crea el teu primer contenidor (Nginx de prova).
5. Esborra'l i crea'l de nou amb Compose.
6. Mou Portainer a Compose.
7. Explora Portainer des del navegador.
8. Crea l'estructura `~/homelab/{compose,data,secrets,scripts}/`.
9. Documenta-ho al `homelab/setup-log.md`.

Capitol 62 — Uptime Kuma: el primer monitor

"Si no saps si el teu servidor està viu o mort, no el pots arreglar. El monitoratge és la base de tot."

62.1 Què aprendràs

- Què és Uptime Kuma.
- Com instal·lar-lo amb Docker Compose.
- Com configurar els teus primers monitors.
- Com rebre alertes per Telegram i correu.
- Com monitorar la pròpia Raspberry.

62.2 Durada estimada

20-30 minuts.

62.3 Què és Uptime Kuma

Uptime Kuma és una eina de monitoratge self-hosted (l'allotges tu). Et permet:

- Fer ping a serveis (HTTP, TCP, ping ICMP, DNS, etc.).
- Veure l'estat en temps real.
- Rebre alertes quan alguna cosa falla.
- Veure gràfiques de latència i temps de resposta.
- Crear pàgines d'estat públiques.

Alternatives:

- **UptimeRobot** (cloud, gratuït fins a 50 monitors).
- **Healthchecks.io** (molt simple, basat en cron).
- **Prometheus + Alertmanager** (molt potent, però més complex — el veurem al cap 67).

Uptime Kuma és el **just mig**: potent però fàcil.

62.4 Instal·lació

Crea `~/homelab/compose/uptime-kuma.yml`:

```
version: "3.8"

services:
  uptime-kuma:
    image: louislam/uptime-kuma:1
```

```
container_name: uptime-kuma
restart: unless-stopped
ports:
  - "3001:3001"
volumes:
  - ./data/uptime-kuma:/app/data
  - /var/run/docker.sock:/var/run/docker.sock:ro
```

Explicació:

- `louislam/uptime-kuma:1` és la imatge oficial. La tag `:1` segueix la darrera 1.x.
- Port 3001 (HTTP).
- Volum `./data/uptime-kuma` per persistir la configuració.
- `docker.sock` perquè Uptime Kuma pugui monitorar contenidors Docker directament.

Engega:

```
cd ~/homelab/compose
docker compose -f uptime-kuma.yml up -d
```

Obre <http://hortosona:3001> al navegador. La primera vegada et demanarà crear un compte d'administrador.

62.5 Configuració inicial

Un cop dins:

1. Crea el compte d'admin.
2. Tria el tema (clar o fosc).
3. Activa 2FA des de "Settings" → "General" → "2FA".

62.6 Crear el primer monitor

A la pantalla principal, fes clic a **"Add New Monitor"**.

Monitorem el **Portainer**:

- **Monitor Type:** HTTPS
- **Friendly Name:** Portainer
- **URL:** <https://localhost:9443>
- **Monitoring Interval:** 60 seconds
- **Retry:** 1
- **Request Timeout:** 10 seconds
- **Accepted Status Codes:** 200-299

Fes clic a **"Save"**. Ara Portainer apareix a la llista amb un punt verd (UP).

62.7 Què monitorar

Afegeix aquests monitors bàsics:

Nom	Tipus	URL/Target
Portainer	HTTPS	https://localhost:9443
Uptime Kuma	HTTP	http://localhost:3001
Raspberry Pi (ping)	Ping	localhost
Grafana (futur)	HTTP	http://localhost:3000
Mosquitto (futur)	TCP	localhost:1883
InfluxDB (futur)	HTTP	http://localhost:8086

Per al ping, has de triar "Ping" com a tipus i posar `localhost` com a target.

62.8 Configurar alertes per Telegram

Això és el que realment val la pena: rebre alertes al mòbil quan alguna cosa falla.

Primer, necessites un **bot de Telegram** (si no el tens):

1. Obre Telegram, parla amb `@BotFather`.
2. Envia `/newbot`.
3. Segueix les instruccions. Et donarà un **token** (una cadena llarga tipus `1234567890:ABCdef...`).
4. Crea un grup amb tu i el bot.
5. Per obtenir el **chat_id**, obre `https://api.telegram.org/bot<TOKEN>/getUpdates` al navegador. Busca el `chat.id` del grup.

A Uptime Kuma:

1. Vés a **Settings** → **Notifications** → **Telegram**.
2. Activa.
3. Posa el **bot token** i el **chat ID**.
4. Fes clic a **"Test"**. Si tot va bé, rebràs un missatge al grup de Telegram.

Torna a **"Add New Monitor"** i configura les notificacions per aquest monitor (a la secció "Notifications" del formulari).

62.9 Configurar alertes per correu

Si vols rebre correus:

1. Settings → Notifications → Email (SMTP).
2. Configura el teu servidor SMTP. Si uses Gmail: - SMTP Host: `smtp.gmail.com` - Port: 587 - Username: el teu correu. - Password: una **app password** de Google (no la teva contrasenya normal). - From i To: el teu correu.

Per obtenir una app password de Gmail:

1. Vés a <https://myaccount.google.com/apppasswords>.
2. Crea una nova "App password" per a "Mail / Uptime Kuma".
3. Google et donarà una contrasenya de 16 caràcters. Usa aquesta.

62.10 Monitorar la pròpia Raspberry

Volem saber si la RPi està viva. Per a això, monitora:

- **Ping** a localhost (o a la seva IP).
- **CPU i RAM** (no és directe a Uptime Kuma — cal un script).
- **Temperatura** (cal un script).

Per a CPU/RAM/temperatura, podem afegir un monitor HTTP que apunti a un petit servidor de mètriques. Però això ho farem al **Cap 67** (Prometheus). Per ara, el ping ja és bona cosa.

També podem monitorar **Uptime Kuma mateix** (un watchdog). Per fer-ho:

1. Crea un monitor de tipus HTTP a <http://localhost:3001>.
2. Si Uptime Kuma es cau, ell mateix no t'avisarà — però el monitor sí.

Això és estrany però útil: si Uptime Kuma es reinicia i funciona, rebràs una alerta de "Recovered".

62.11 Pàgina d'estat pública

Uptime Kuma permet crear una pàgina d'estat pública:

1. Settings → Status Page.
2. Crea una nova pàgina.
3. Afegeix els monitors que vols que siguin visibles.
4. Tria un nom (per exemple, "BernatLab Status").
5. Opcional: posa una contrasenya perquè només tu la puguis veure.

La pàgina estarà disponible a <http://hortosona:3001/status/bernatlab> o similar.

Això és útil si vols compartir amb algú l'estat del sistema sense donar accés a la consola d'admin.

62.12 Què fer quan arriba una alerta

Quan Uptime Kuma t'avisa per Telegram que un servei està caigut:

1. **No entris en pànic.** És normal que un servei caigui de tant en tant.
2. **Mira els logs:** a Portainer, selecciona el contenidor i mira els logs.
3. **Comprova si és transitori:** espera 5 minuts. Si torna sol, era un reinici de Docker o de la RPi.
4. **Si persisteix:** consulta el runbook pertinent (cap 68).
5. **Si no hi ha runbook:** consulta el cap 56 (diagnòstic).

62.13 Què ve després

Ja tens Docker, Portainer i Uptime Kuma. Al **Cap 63** muntarem la cadena de dades: **Mosquitto (MQTT)**, **InfluxDB** i **Grafana**. Això és el cor de l'hort.

62.14 Errors habituals

Error 1: el contenidor no arrenca.

Mira els logs:

```
docker logs uptime-kuma
```

Sovint és un conflicte de port o de volum.

Error 2: Telegram no envia.

Comprova que el token i el chat ID són correctes. Assegura't que el bot és membre del grup. Fes servir el botó "Test".

Error 3: monitors falsos positius.

Si la teva xarxa és inestable, rebràs alertes que no són reals. Augmenta l'interval de monitoratge o el "Retry" a 3 per evitar-ho.

Error 4: pàgina d'estat inaccessible.

Assegura't que el port 3001 és accessible. Si tens UFW, permet-ho per a la xarxa local.

62.15 Resum

Uptime Kuma és la teva primera línia de defensa: si un servei cau, t'avisa. Hem vist:

- Instal·lació amb Compose.
- Configuració de monitors.
- Alertes per Telegram i correu.
- Pàgina d'estat pública.

Al **Cap 63** muntarem la cadena de dades de l'hort.

62.16 Exercicis pràctics

1. Instal·la Uptime Kuma amb Compose.
2. Crea el compte d'admin i activa 2FA.
3. Crea 5 monitors (Portainer, Uptime Kuma, ping, i 2 més).
4. Configura alertes per Telegram.
5. Configura alertes per correu.
6. Crea una pàgina d'estat pública.
7. Documenta els monitors al `home/lab/setup-log.md`.

Capítol 63 — La cadena de dades: MQTT, InfluxDB, Grafana

"Aquest capítol és el cor de l'hort. Tot el que vingui després (nodes LoRa, sensors, automatitzacions) acaba passant per aquí."

63.1 Què aprendràs

- Què és MQTT i per què és el protocol estàndard per a IoT.
- Com instal·lar **Mosquitto** (broker MQTT).
- Com instal·lar **InfluxDB** (base de dades de sèries temporals).
- Com instal·lar **Telegraf** (ponte entre MQTT i InfluxDB).
- Com instal·lar **Grafana** (visualització).
- Com veure les primeres dades a Grafana.

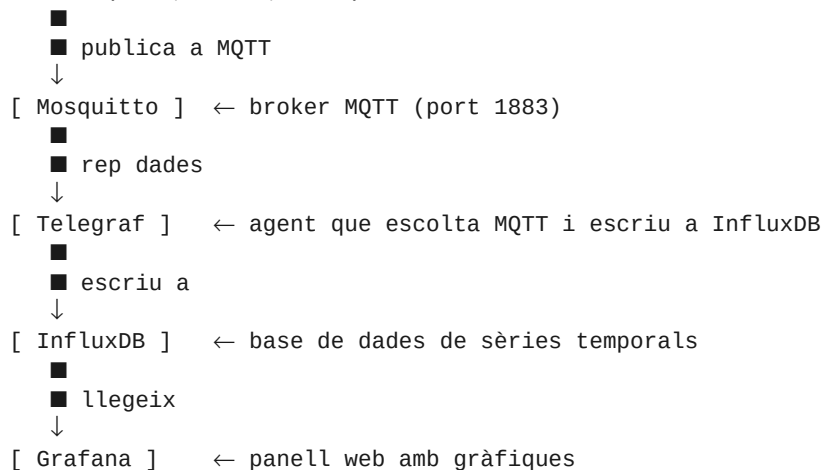
63.2 Durada estimada

1-1.5 hores.

63.3 L'arquitectura de dades

Abans de tocar res, entenem què construirem:

Sensor (LoRa, ESP32, etc.)



Aquesta és la cadena estàndard per a IoT. Funciona.

Alternatives:

- **InfluxDB 1.x** amb el seu propi plugin MQTT (sense Telegraf). Més simple però menys flexible.
- **MQTT** → **Node-RED** → **InfluxDB** → **Grafana**. Node-RED pot fer la part de Telegraf, però afegeix complexitat.

- **MQTT** → **base de dades SQL** → **Grafana**. Més potència per a consultes, menys per a sèries temporals.

La meva recomanació: **Telegraf** per la seva senzillesa i perquè és part de la stack InfluxData (els mateixos que fan InfluxDB).

63.4 Instal·lar Mosquitto

Crea `~/homelab/secrets/mqtt.env`:

```
MQTT_USER=bernats
MQTT_PASSWORD=una-contrasenya-forta-de-32-caracters
```

Mai posis la contrasenya directament al docker-compose.

Crea `~/homelab/compose/mqtt.yml`:

```
version: "3.8"

services:
  mosquitto:
    image: eclipse-mosquitto:2
    container_name: mosquitto
    restart: unless-stopped
    ports:
      - "1883:1883"
      - "9001:9001"
    volumes:
      - ./data/mosquitto/data:/mosquitto/data
      - ./data/mosquitto/log:/mosquitto/log
      - ./mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf
      - ./mosquitto/passwd:/mosquitto/config/passwd
    env_file:
      - ../secrets/mqtt.env
```

Crea `~/homelab/compose/mosquitto/mosquitto.conf`:

```
# Configuració bàsica de Mosquitto
persistence true
persistence_location /mosquitto/data/
log_dest stdout

# Autenticació
allow_anonymous false
password_file /mosquitto/config/passwd

# Listener MQTT
listener 1883
protocol mqtt

# Listener WebSockets (opcional, per accedir des del navegador)
listener 9001
protocol websockets
```

Crea el fitxer de contrasenyes:

```
cd ~/homelab/compose
docker run --rm -it \
  -v $(pwd)/mosquitto/passwd:/mosquitto/config/passwd \
  eclipse-mosquitto:2 \
  mosquitto_passwd -c /mosquitto/config/passwd $MQTT_USER
```

(Substitueix `$MQTT_USER` pel valor del `.env`, o passa'l manualment.)

Engega:

```
cd ~/homelab/compose
docker compose -f mqtt.yml up -d
```

63.5 Provar Mosquitto

Des del teu ordinador, publica un missatge:

```
# Publicar
mosquitto_pub -h hortosona -p 1883 \
  -u bernat -P 'contrasenya' \
  -t 'test/hort' -m 'Hola des del BernatLab!'

# Subscriure (en un altre terminal)
mosquitto_sub -h hortosona -p 1883 \
  -u bernat -P 'contrasenya' \
  -t 'test/#' -v
```

Si veus el missatge al segon terminal, tot funciona.

63.6 Instal·lar InfluxDB

Crea ~/homelab/secrets/influxdb.env:

```
DOCKER_INFLUXDB_INIT_MODE=setup
DOCKER_INFLUXDB_INIT_USERNAME=admin
DOCKER_INFLUXDB_INIT_PASSWORD=una-altra-contrasenya-forta
DOCKER_INFLUXDB_INIT_ORG=bernatlab
DOCKER_INFLUXDB_INIT_BUCKET=hort
DOCKER_INFLUXDB_INIT_RETENTION=1y
DOCKER_INFLUXDB_INIT_ADMIN_TOKEN=un-token-llarg-aleatori-de-32-chars
```

Crea ~/homelab/compose/influxdb.yml:

```
version: "3.8"

services:
  influxdb:
    image: influxdb:2
    container_name: influxdb
    restart: unless-stopped
    ports:
      - "8086:8086"
    volumes:
      - ./data/influxdb:/var/lib/influxdb2
    env_file:
      - ../secrets/influxdb.env
```

Engega:

```
cd ~/homelab/compose
docker compose -f influxdb.yml up -d
```

Obre <http://hortosona:8086> al navegador. Hauries de veure la pantalla de benvinguda. Ja està configurat (les variables d'entorn ho fan).

63.7 Instal·lar Telegraf

Telegraf escolta MQTT i escriu a InfluxDB.

Crea ~/homelab/compose/telegraf.conf:

```
[agent]
interval = "30s"
flush_interval = "10s"

[[outputs.influxdb_v2]]
urls = ["http://influxdb:8086"]
token = "${INFLUX_TOKEN}"
org = "bernatlab"
bucket = "hort"

[[inputs.mqtt_consumer]]
servers = ["tcp://mosquitto:1883"]
topics = ["sensors/#", "bernatlab/#"]
qos = 0
username = "${MQTT_USER}"
password = "${MQTT_PASSWORD}"
data_format = "json"
tag_keys = ["node", "sensor"]
json_string_fields = ["value"]
```

Crea ~/homelab/compose/telegraf.yml:

```
version: "3.8"

services:
  telegraf:
    image: telegraf:1.30
    container_name: telegraf
    restart: unless-stopped
    depends_on:
      - mosquitto
      - influxdb
    volumes:
      - ./telegraf.conf:/etc/telegraf/telegraf.conf:ro
    environment:
      - INFLUX_TOKEN=${DOCKER_INFLUXDB_INIT_ADMIN_TOKEN}
      - MQTT_USER=${MQTT_USER}
      - MQTT_PASSWORD=${MQTT_PASSWORD}
    env_file:
      - ../secrets/mqtt.env
      - ../secrets/influxdb.env
```

Engega:

```
cd ~/homelab/compose
docker compose -f telegraf.yml up -d
```

Verifica els logs:

```
docker logs telegraf
```

Si tot va bé, veuràs línies tipus:

```
2026-07-09T12:00:00Z I! Loaded inputs: mqtt_consumer
2026-07-09T12:00:00Z I! Loaded aggregators:
2026-07-09T12:00:00Z I! Loaded processors:
2026-07-09T12:00:00Z I! Loaded outputs: influxdb_v2
```

63.8 Instal·lar Grafana

Crea ~/homelab/compose/grafana.yml:

```
version: "3.8"

services:
  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    restart: unless-stopped
    ports:
      - "3000:3000"
    volumes:
      - ./data/grafana:/var/lib/grafana
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_PASSWORD}
      - GF_USERS_ALLOW_SIGN_UP=false
    env_file:
      - ../secrets/grafana.env
```

Crea `~/homelab/secrets/grafana.env`:

```
GRAFANA_PASSWORD=una-contrasenya-forta
```

Engega:

```
cd ~/homelab/compose
docker compose -f grafana.yml up -d
```

Obre <http://hortosona:3000>. Login: `admin` / la teva contrasenya. Canvia la contrasenya al primer login.

63.9 Connectar Grafana a InfluxDB

A Grafana:

1. **Connections** → **Data sources** → **Add data source**.
2. Tria **InfluxDB**.
3. Configura: - URL: <http://influxdb:8086> - Organization: `bernat lab` - Token: el que has posat al `.env` d'`InfluxDB` - Default bucket: `hort`
4. **Save & test**. Si tot va bé, veuràs "Data source is working".

63.10 Provar la cadena sencera

Des de la terminal:

```
mosquitto_pub -h hortosona -p 1883 \
  -u bernat -P 'contrasenya' \
  -t 'sensors/hort1/temperatura' \
  -m '{"node":"hort1","sensor":"temperatura","value":23.5,"unit":"C"}'
```

A Grafana:

1. **Explore** (la icona de brúixola a l'esquerra).
2. Tria el data source `InfluxDB`.
3. Escriu una query com `SELECT mean("value") FROM "hort" WHERE ("sensor" = 'temperatura') GROUP BY time($__interval), node fill(null)`.

4. Veuràs un punt a la gràfica.

Si veus el punt, **tens la cadena sencera funcionant**: del teu terminal a una gràfica a Grafana, passant per MQTT, Telegraf i InfluxDB.

63.11 El primer panell

Crea un dashboard:

1. + → **Dashboard** → **Add visualization**.
2. Tria InfluxDB.
3. Configura: - **From**: `hort` - **Measurement**: `mqtt_consumer` (Telegraf usa aquest nom per defecte). - **Field**: `value`. - **Filters**: `sensor = temperatura`.
4. Tria el tipus de visualització (Time series).
5. Desa el panell.
6. Desa el dashboard.

Ara cada cop que algú publiqui una temperatura a `sensors/hort1/temperatura`, apareixerà al panell.

63.12 Com organitzar les dades

A l'hora de publicar a MQTT, tria un esquema de temes clar. Jo uso:

- `sensors/<ubicacio>/<sensor>` per a dades de sensors.
- `bernatlab/<servei>/<accio>` per a esdeveniments del sistema.
- `alerts/<severitat>/<servei>` per a alertes.

Exemple:

- `sensors/hort1/temperatura` → temperatura de l'hort 1.
- `sensors/hort1/humitat` → humitat de l'hort 1.
- `sensors/ciutat/temperatura` → temperatura al sensor de la ciutat.
- `bernatlab/mqtt/disconnect` → esdeveniment de desconnexió.
- `alerts/critical/grafana` → alerta crítica a Grafana.

Això et permetrà filtrar fàcilment a Grafana i Node-RED.

63.13 Què ve després

Ja tens la cadena de dades. Al **Cap 64** afegirem **Node-RED**, que ens permetrà fer automatitzacions reals (si la temperatura baixa de 5°C, encén la calefacció de l'hivernacle).

63.14 Errors habituals

Error 1: "Connection refused" a MQTT.

Comprova que el port 1883 està exposat i que el contenidor està en marxa.

Error 2: Telegraf no escriu a InfluxDB.

Mira els logs de Telegraf. Sovint és un token mal copiat o un nom de bucket incorrecte.

Error 3: Grafana no es connecta a InfluxDB.

Comprova la URL, el token i el nom d'organització. El "Save & test" t'ho dirà.

Error 4: les dades arriben però Grafana no les mostra.

El tema de la query és delicat. Comença amb `SELECT * FROM "mqtt_consumer" LIMIT 10` per veure què hi ha, i refina després.

63.15 Resum

Ara tens la cadena de dades completa:

- **Mosquitto** (MQTT) escolta publicacions.
- **Telegraf** les tradueix i les envia a InfluxDB.
- **InfluxDB** les emmagatzema.
- **Grafana** les visualitza.

Això és la base sobre la qual vindrà tot: nodes LoRa, sensors, alertes, automatitzacions. La combinació MQTT + InfluxDB + Grafana és el cor de l'IoT modern.

63.16 Exercicis pràctics

1. Instal·la Mosquitto, InfluxDB, Telegraf i Grafana.
2. Configura l'autenticació a Mosquitto.
3. Publica una dada de prova.
4. Connecta Grafana a InfluxDB.
5. Crea el teu primer panell.
6. Publica 5 dades de prova amb valors diferents.
7. Afegeix un monitor d'Uptime Kuma per a Grafana i Mosquitto.
8. Documenta els temes MQTT que has triat.

Capitol 64 — Node-RED: les primeres automatitzacions

"Node-RED és un llenguatge visual. En lloc d'escriure codi, arrossegues caixes. Per a automatitzacions de l'hort, és perfecte."

64.1 Què aprendràs

- Què és Node-RED i per què serveix.
- Com instal·lar Node-RED amb Docker.
- Com fer el teu primer flow.
- Com escoltar MQTT des de Node-RED.
- Com publicar a MQTT des de Node-RED.
- Com fer una automatització condicional real.
- Com desar els teus flows.

64.2 Durada estimada

1-1.5 hores.

64.3 Què és Node-RED

Node-RED és una eina de programació visual, basada en Node.js, creada per IBM. És com Scratch, però per a IoT.

Pensa en ell com un "IFTTT professional":

- Rep esdeveniments d'un costat (MQTT, HTTP, temporitzador...).
- Hi aplica una lògica (if/else, transformacions...).
- Fa alguna cosa (publicar a MQTT, enviar un correu, encendre un relé...).

Tot arrossegant caixes. **Sense escriure una línia de codi** (tot i que pots fer-ho si vols).

Exemples d'ús:

- Si la temperatura de l'hort baixa de 5°C, encén una estufa.
- Si el nivell d'humitat baixa del 30%, rega.
- Si el node LoRa no ha enviat dades en 1 hora, avisa per Telegram.
- Cada dia a les 20h, envia un resum a Telegram.

Node-RED és **l'eina perfecta** per a automatitzacions de l'hort.

64.4 Instal·lació

Crea `~/homelab/compose/node-red.yml`:

```
version: "3.8"

services:
  node-red:
    image: nodered/node-red:latest
    container_name: node-red
    restart: unless-stopped
    ports:
      - "1880:1880"
    volumes:
      - ./data/node-red:/data
    environment:
      - TZ=Europe/Madrid
    depends_on:
      - mosquitto
```

Engrega:

```
cd ~/homelab/compose
docker compose -f node-red.yml up -d
```

Obre <http://hortosona:1880> al navegador. Hauries de veure l'editor visual.

64.5 Interfície de Node-RED

A la part esquerra tens la **paleta de nodes**, organitzats per categories:

- **input**: nodes que reben dades (MQTT, HTTP, timer, inject...).
- **output**: nodes que envien dades (MQTT, HTTP, email, file...).
- **function**: nodes per fer lògica (if/else, switch, change, function...).
- **social**: nodes per a serveis externs (Telegram, Twitter, email...).
- **storage**: nodes per emmagatzematge (SQL, InfluxDB...).

Al centre tens el **flow**, on arrossegues els nodes.

A la dreta tens la **informació del node** i el **debug**.

64.6 El primer flow: "Hello, world!"

Arrossega un node `inject` al flow.

1. Configura'l: posa'l una mica a la dreta, fes doble clic, i tria "Time stamp" com a payload.
2. Arrossega un node `debug`.
3. Connecta'ls: arrossega des del cercle gris de la dreta del `inject` fins al cercle gris de l'esquerra del `debug`.
4. Fes clic al botó blau **Deploy** (a dalt a la dreta).
5. Fes clic al botó quadrat blau a la dreta del node `inject` (per activar-lo).

A la dreta, a la pestanya **debug**, hauries de veure l'hora actual.

Això és el teu primer flow. Simplement injecta un valor (l'hora) i l'envia a debug. Ara complica-ho.

64.7 Escoltar dades MQTT

Arrossega un node `mqtt in` (a la categoria "network"):

- Fes doble clic.
- **Server:** clic a la icona del llapis per afegir un servidor nou.
- **Name:** BernatLab MQTT
- **Server:** `mosquitto` (o `hortosona`)
- **Port:** 1883
- **Security:** activa username/password i posa les credencials de Mosquitto.
- **Action:** "Subscribe to topic"
- **Topic:** `sensors/#`
- **QoS:** 0

Connecta'l a un node `debug` (canvia el node debug a "complete msg object" per veure tot).

Deploy. Ara cada cop que algú publiqui a `sensors/...`, ho veuràs al debug.

Si tens dades a MQTT, ja les veuràs aquí.

64.8 Publicar a MQTT

Per publicar:

- Arrossega un node `mqtt out`.
- Configura'l igual que el `mqtt in` (mateix servidor, mateixa autenticació).
- **Action:** "Publish to topic".
- **Topic:** `bernatlab/test/hola`.

Per enviar un missatge, connecta'l a algun node d'input (per exemple, un `inject` amb payload "Hola des de Node-RED").

64.9 La primera automatització: alerta per temperatura baixa

Farem un flow que:

1. Escolta `sensors/hort1/temperatura`.
2. Si el valor és $< 5^{\circ}\text{C}$, publica una alerta a `alerts/critical/hort`.
3. Si no, no fa res.

Passos:

1. Arrossega un `mqtt in` configurat com abans, subscrit a `sensors/hort1/temperatura`.
2. Arrossega un node `function` i posa-hi:

```
// El payload arriba com a string JSON
const msg = JSON.parse(msg.payload);
const temperatura = msg.value;

if (temperatura < 5) {
  msg.payload = {
    node: msg.node,
    sensor: msg.sensor,
    value: temperatura,
    message: "Risc de gelada!",
  };
  return msg;
}

// No enviem res si no és una alerta
return null;
```

1. Arrossega un `mqtt out` configurat per publicar a `alerts/critical/hort`.
2. Connecta'ls en cadena: `mqtt in` → `function` → `mqtt out`.
3. Deploy.

Prova publicant una temperatura baixa:

```
mosquitto_pub -h hortosona -p 1883 -u bernat -P 'contrasenya' \
  -t 'sensors/hort1/temperatura' \
  -m '{"node":"hort1","sensor":"temperatura","value":2.3,"unit":"C"}'
```

Mira els logs de Mosquitto o subscriu-te a `alerts/#` per veure l'alerta.

64.10 La segona automatització: regar si la humitat baixa

Aquest és el cas clàssic de l'hort:

1. Escolta `sensors/hort1/humitat` periòdicament.
2. Si humitat < 30%, publica una ordre a `bernat lab/reg/on` durant 5 minuts.
3. Envia una alerta per Telegram.

Això ja és més complex. Ho fem pas a pas.

Part 1: escoltar la humitat

Node `mqtt in` a `sensors/hort1/humitat`.

Part 2: comparar amb el llindar

Node `function`:

```
const data = JSON.parse(msg.payload);
const humitat = data.value;
const node = data.node;

if (humitat < 30) {
  msg.payload = {
    node: node,
    humitat: humitat,
    action: "regar",
    duration: 300, // 5 minuts en segons
  };
  return msg;
}
```

```
return null;
```

Part 3: enviar ordre al sistema de reg

Aquí hem de decidir com és el sistema de reg. En el M3 parlàvem d'un node LoRa amb relés. Per ara, simulem:

Node `mqtt out` a `bernatlab/reg/hort1/on` amb el payload generat.

Part 4: alerta per Telegram

Aquí veurem al **Cap 66** com configurar el bot de Telegram. Per ara, podem posar un node `debug` per veure què passaria.

64.11 Com desar els flows

Els flows es desen automàticament quan fas Deploy. Però hi ha més:

- **Exportar:** pots exportar un flow com a JSON (menú → Export).
- **Importar:** pots importar un flow d'un altre (menú → Import).
- **Subversion:** els flows estan a `./data/node-red/flows.json`. Pots versionar aquest fitxer a Git.

Jo recomano **versionar el `flows.json`** al repo del BernatLab. Així tens un historial de canvis i pots recuperar fàcilment.

64.12 La paleta de nodes

A mesura que necessitis més coses, pots afegir nodes nous. Node-RED té un sistema de "palette manager":

1. Menú → **Manage palette**.
2. **Install**.
3. Busca el node que vulguis.

Alguns nodes útils per a l'hort:

- `node-red-node-ui-table`: per visualitzar dades en taules.
- `node-red-dashboard`: per crear un panell web per a l'hort.
- `node-red-contrib-modbus`: per comunicar amb dispositius Modbus.
- `node-red-contrib-influxdb`: per escriure directament a InfluxDB des de Node-RED.

64.13 Què ve després

Node-RED és la peça que fa servir **totes les altres**. Al **Cap 65** afegirem un **node LoRa** real al camp, que publicarà dades a MQTT, i les veuràs a Grafana gràcies a aquesta cadena.

64.14 Errors habituals

Error 1: "Connection refused" a MQTT.

Comprova que el servidor, port, usuari i contrasenya són correctes.

Error 2: el flow no es desplega.

Mira l'error a la consola (a sota). Sovint és un node mal configurat o un JSON invàlid al node `function`.

Error 3: la funció no retorna res.

Si la funció retorna `null` o no retorna, el flow s'atura. Assegura't que el flux té un camí per a tots els casos.

Error 4: les dades no es reben.

Comprova que el topic MQTT és correcte. Un `sensors/#` rep `sensors/hort1/temperatura`, però un `sensors/hort1/#` no rep `sensors/hort2/temperatura`.

64.15 Resum

Node-RED és la cola del BernatLab. Aquí és on la lògica esdevé visual i intuïtiva. Hem vist:

- Instal·lació amb Compose.
- Interfície i paleta de nodes.
- Com escoltar i publicar a MQTT.
- Com fer una funció que filtra i transforma missatges.
- Com fer una automatització condicional.

Al **Cap 65** afegirem un node LoRa al camp, que serà el primer consumidor real d'aquesta cadena.

64.16 Exercicis pràctics

1. Instal·la Node-RED.
2. Fes el "Hello, world!" amb un `inject` i un `debug`.
3. Connecta't a Mosquitto des de Node-RED.
4. Publica una temperatura i rep-la a Node-RED.
5. Fes una funció que filtri temperatures $< 5^{\circ}\text{C}$.
6. Fes una automatització que publiqui una alerta a `alerts/#` quan la humitat $< 30\%$.
7. Exporta el teu flow a JSON i guarda'l al repo.
8. Documenta els flows al `homelab/setup-log.md`.

Capítol 65 — El node LoRa al camp

*“Aquest capítol és on l'hort es connecta de veritat. Fins ara tenies un servidor; ara tens un sistema que escolta el que passa al camp.”**

65.1 Què aprendràs

- Quin hardware necessites per al node LoRa.
- Com muntar el node (ESP32 + SX1262 + sensors).
- Com programar-lo per publicar dades a MQTT.
- Com rebre les dades a la Raspberry.
- Com veure les primeres dades reals a Grafana.

65.2 Durada estimada

2-3 hores (comptant proves).

65.3 Quin hardware

Per al node del camp:

- **ESP32-WROOM-32** (la placa de microcontrol·lador): 5 €.
- **Mòdul SX1262 868 MHz** (la ràdio LoRa): 10 €.
- **Antena 868 MHz** amb connector IPEX/U.FL: 3 €.
- **Sensor BME280** (temperatura, humitat, pressió): 5 €.
- **Sensor d'humitat del sòl capacitiu**: 3 €.
- **Cable Dupont** (per connectar tot): 5 €.
- **Caixa estanca IP65**: 5 €.
- **Bateria 18650 + carregador TP4056**: 8 €.
- **Placa solar 6V 1W** (opcional): 8 €.

Total: ~50 € per node.

Si ja tens un gateway LoRaWAN (Dragino LPS8, RAK7258, etc.), el node serà més simple — parla directament amb TTN, no necessita un gateway local.

65.4 Decidir: LoRaWAN o P2P

Al **Cap 25** ja vam decidir això. Recordatori breu:

- **LoRaWAN + TTN**: el node parla amb un gateway, TTN redirigeix a un servidor (el teu, via MQTT). Molt escalable, però depens d'un servei extern.

- **LoRa P2P**: dos mòduls SX1262 parlen directament. No hi ha servidor intermedi. Més simple, però menys escalable.

Per a un homelab amb un sol node, **P2P és més simple** — no necessites TTN ni un gateway, només dos SX1262. Però **LoRaWAN + un gateway Dragino LPS8** és més robust.

Aquest capítol assumeix **LoRaWAN amb un gateway local**. Si vols P2P, els canvis són petits (mira el Cap 31).

65.5 Muntar el gateway (si no el tens)

El gateway és un dispositiu que escolta els nodes LoRa i envia les dades a un servidor. El model més popular per a homelab és el **Dragino LPS8**.

Per muntar-lo:

1. Connecta'l al router via Ethernet.
2. Connecta'l a l'antena (port SMA).
3. Endolla'l.
4. Accedeix a la interfície web (per defecte, <http://192.168.1.77>).
5. Configura'l per parlar amb TTN o amb el teu servidor local.

Al cap 27 explicàvem com configurar el gateway amb concentratord. Si tens dubtes, mira'l.

65.6 Muntar el node

Connexions de l'ESP32 al SX1262 (SPI):

ESP32	SX1262
3.3V	VCC
GND	GND
GPIO 5	SCK
GPIO 27	MISO
GPIO 2	MOSI
GPIO 4	NSS (CS)
GPIO 14	RST
GPIO 26	DIO1
GPIO 33	BUSY

I els sensors:

ESP32	BME280	Humitat sòl
3.3V	VCC	VCC
GND	GND	GND

ESP32	BME280	Humitat sòl
GPIO 21 (SDA)	SDA	-
GPIO 22 (SCL)	SCL	-
GPIO 34 (ADC)	-	AOUT

Això és la part elèctrica. Ara ve la programació.

65.7 Programar el node

Usa **Arduino IDE** o **PlatformIO** per programar l'ESP32. Jo recomano PlatformIO (integrat a VS Code) per la seva potència.

Crea un nou projecte "ESP32 + LoRa + BME280".

Llibreries necessàries (via PlatformIO):

```
lib_deps =
    sandeepmistry/LoRa@^0.8.0
    adafruit/Adafruit BME280 Library@^2.2.4
    bblanchon/ArduinoJson@^7.0.0
```

Codi bàsic:

```
#include <SPI.h>
#include <LoRa.h>
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME280.h>

#define SS_PIN    4
#define RST_PIN   14
#define DI00_PIN  26

Adafruit_BME280 bme;

int counter = 0;

void setup() {
    Serial.begin(115200);

    // Setup LoRa
    LoRa.setPins(SS_PIN, RST_PIN, DI00_PIN);
    if (!LoRa.begin(868E6)) {
        Serial.println("Error iniciant LoRa");
        while (1);
    }
    LoRa.setSpreadingFactor(7);
    LoRa.setSignalBandwidth(125E3);
    LoRa.setTxPower(14);

    // Setup BME280
    if (!bme.begin(0x76)) {
        Serial.println("Error iniciant BME280");
        while (1);
    }
}

void loop() {
    // Llegeix sensors
```



```

float temperatura = bme.readTemperature();
float humitat = bme.readHumidity();
float pressio = bme.readPressure() / 100.0;

// Llegeix humitat del sòl (0-4095, menor = més humit)
int soilRaw = analogRead(34);
float soilPercent = map(soilRaw, 0, 4095, 100, 0); // invertir

// Crea el payload
String payload = "{";
payload += "\"node\":\"hort1\",";
payload += "\"counter\":\"" + String(counter) + ",";
payload += "\"temperatura\":\"" + String(temperatura, 1) + ",";
payload += "\"humitat\":\"" + String(humitat, 1) + ",";
payload += "\"pressio\":\"" + String(pressio, 1) + ",";
payload += "\"soil\":\"" + String(soilPercent, 1);
payload += "}";

// Envia via LoRa
LoRa.beginPacket();
LoRa.print(payload);
LoRa.endPacket();

Serial.println("Enviat: " + payload);
counter++;

// Espera 5 minuts
delay(300000);
}

```

Pujem el codi a l'ESP32.

65.8 Receptar les dades a la Raspberry

Al gateway Dragino, configura el "Packet Forwarder" per enviar les dades a un servidor local:

1. Accedeix a la interfície web del gateway.
2. Configura el servidor primari com a `localhost` (o la IP de la RPi si el gateway és un altre dispositiu).
3. Configura el port 1700 (per defecte del packet forwarder).

A la Raspberry, hem de tenir un servei que escolti el port 1700 i enviï les dades a MQTT. Una opció és **ChirpStack** (molt potent però complex), o **golismo** (molt simple).

Alternativa: usa **TTN** com a intermediari. El gateway envia a TTN, i TTN redirigeix a un servidor MQTT (que pot ser la teva RPi).

65.9 Configurar TTN (mètode recomanat)

The Things Network (TTN) és gratuït per a ús personal. Configurar-lo:

1. Crea un compte a <https://www.thethingsnetwork.org>.
2. Crea una **Application**.
3. Crea un **End device** dins l'aplicació: - Manufacturer: el teu (o selecciona el perfil correcte). - DevEUI, AppEUI, AppKey: els que tingui el teu node (o genera'ls automàticament).

4. Configura el **Payload formatter** (Uplink) per desxifrar CayenneLPP o el teu format JSON.
5. Configura l'**integració MQTT**: - Default endpoint. - Porta 1883. - Username i password: els de Mosquitto (però TTN ha de poder accedir — cal exposar Mosquitto, cosa que NO recomanem per seguretat).

Alternativa més segura: usa TTN per desar les dades, i un script a la RPi que les vagi a buscar periòdicament via l'API de TTN (no via MQTT directe).

65.10 Receptar amb un script Python local

Si vols evitar TTN, pots tenir un script Python a la RPi que escolti el port 1700:

Crea `~/homelab/scripts/lora-listener.py`:

```
import socket
import json
import paho.mqtt.client as mqtt
import os

# Configuració
TTN_MQTT_USER = "bernats"
TTN_MQTT_PASSWORD = os.environ["MQTT_PASSWORD"]
MQTT_BROKER = "localhost"
MQTT_PORT = 1883

# Client MQTT
client = mqtt.Client()
client.username_pw_set(TTN_MQTT_USER, TTN_MQTT_PASSWORD)
client.connect(MQTT_BROKER, MQTT_PORT, 60)

# Socket per escoltar el gateway
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind(("0.0.0.0", 1700))

print("Escoltant al port 1700...")

while True:
    data, addr = sock.recvfrom(4096)
    # Parseja el protocol TTN Packet Forwarder
    # (Aquí va una mica de feina per deserialitzar el format)
    # Publica a MQTT
    payload = {"raw": data.hex()}
    client.publish("lora/gateway/raw", json.dumps(payload))
```

Aquest script és esquemàtic — caldria implementar correctament el protocol del packet forwarder. En la pràctica, és molt més fàcil usar TTN.

65.11 Veure les primeres dades a Grafana

Un cop el node envia, el gateway rep, TTN redirigeix, i la teva cadena MQTT → InfluxDB → Grafana ja té les dades.

Si tot està connectat:

1. Publica una dada de prova (o espera que el node enviï).
2. Mira els logs de Mosquitto: `docker logs mosquitto`.
3. Mira els logs de Telegraf: `docker logs telegraf`.

4. A Grafana, crea una gràfica nova amb `SELECT mean("temperatura") FROM "mqtt_consumer" WHERE node = 'hort1'`.

Si veus una línia a la gràfica, **tot funciona**. El teu hort ara és al núvol.

65.12 Bateria i durada

Si el node funciona amb bateria, el consum és crític. Optimitzacions:

- **Deep sleep** entre transmissions (ESP32 consumeix 10 µA en sleep).
- Transmetre cada 5-15 minuts en lloc de cada minut.
- Usar SF7 (ràpida, menys temps d'aire).
- Reduir la potència de transmissió al mínim necessari.

Codi per afegir deep sleep:

```
#include <esp_sleep.h>

void loop() {
  // Llegeix sensors
  // Envia dades
  Serial.println("Anant a dormir 5 minuts");
  esp_sleep_enable_timer_wakeup(300 * 1000000);
  esp_deep_sleep_start();
}
```

Així l'ESP32 es desperta cada 5 minuts, llegeix sensors, envia, i torna a dormir. La bateria pot durar setmanes o mesos.

65.13 Què ve després

Ja tens el node LoRa enviant dades reals des del camp. Al **Cap 66** afegirem un **bot de Telegram** per rebre alertes al mòbil.

65.14 Errors habituals

Error 1: el node no es connecta al gateway.

Comprova:

- Clau d'aplicació (AppKey) correcta.
- Freqüència correcta (868 MHz a Europa).
- El node està dins del rang del gateway.
- La DevEUI és correcta a TTN.

Error 2: les dades arriben però no es veuen a Grafana.

Comprova:

- Telegraf està escoltant el topic correcte.
- InfluxDB està rebent dades (botó "Query" a InfluxDB UI).
- La query de Grafana té el bon nom de mesura (`mqtt_consumer` per defecte).

Error 3: el node es queda sense bateria ràpid.

Augmenta l'interval de transmissió, activa deep sleep, redueix la potència de transmissió.

Error 4: senyals dèbils (RSSI baix).

Afegeix una antena millor, canvia la ubicació del node, augmenta SF (a costa de més temps d'aire i consum).

65.15 Resum

Ara tens el node LoRa enviant dades reals a través de TTS → Mosquitto → Telegraf → InfluxDB → Grafana. La cadena completa funciona. A partir d'aquí, el que pots fer és infinit:

- Afegir més nodes.
- Afegir més sensors (vent, pluja, radiació solar).
- Fer automatitzacions a Node-RED basades en les dades.
- Afegir alertes quan els valors surtin dels llindars.

65.16 Exercicis pràctics

1. Munta el node (ESP32 + SX1262 + sensors).
2. Programa'l amb el codi bàsic.
3. Configura el gateway (o usa TTN).
4. Configura Telegraf per escoltar `lora/+/+`.
5. Crea un panell a Grafana amb les dades.
6. Activa deep sleep al node.
7. Col·loca el node a l'hort (o al balcó) i espera 24 h.
8. Documenta la configuració al `home lab/setup-log.md`.

Capitol 66 — Bot de Telegram: alertes al mòbil

"Rebre una alerta al mòbil quan passa alguna cosa a l'hort, en temps real, és el salt entre un sistema que recull dades i un sistema que et cuida."

66.1 Què aprendràs

- Com crear un bot de Telegram.
- Com obtenir el chat_id.
- Com enviar missatges des de la RPi.
- Com configurar alertes automàtiques des de Node-RED.
- Com combinar-ho amb Uptime Kuma.
- Bones pràctiques per a bots de Telegram.

66.2 Durada estimada

30-45 minuts.

66.3 Per què Telegram

Telegram és ideal per a alertes perquè:

- **Gratuït** per a bots.
- **API oberta** i ben documentada.
- **Grups** amb múltiples membres (perfecte per compartir l'estat amb família).
- **Markdown** i **HTML** als missatges.
- **Sense límit** pràctic de missatges.
- **Multiplataforma**: iOS, Android, web, desktop.

Alternatives:

- **Signal**: més segur però l'API per a bots és menys popular.
- **Discord**: similar però pensat per a comunitats.
- **Pushover**, **Pushbullet**, **Ntfy**: serveis específics per a notificacions.
- **SMS**: car i no xifrat.

Telegram és la millor elecció per a un homelab personal.

66.4 Crear el bot

Al teu mòbil o web de Telegram:

1. Cerca **@BotFather**.
2. Inicia conversa.
3. Envia `/newbot`.
4. Tria un nom (per exemple, "BernatLab Bot").
5. Tria un username (ha d'acabar en `bot`, per exemple, "bernatlab_alerts_bot").
6. Rebràs un **token**. Guarda'l al gestor de contrasenyes.

Exemple de token: `1234567890:ABCdefGHIjklMNOpqrsTUVwxyz123456`.

66.5 Crear el grup

1. Crea un grup a Telegram anomenat "BernatLab" (o com vulguis).
2. Afegeix el bot al grup.
3. Per obtenir el **chat_id** del grup: - Com a admin, obre <https://api.telegram.org/bot<TOKEN>/getUpdates>. - Cerca el `chat_id` del grup (un número negatiu, ex: `-1001234567890`). - Guarda'l al gestor.

66.6 El primer missatge des de la RPi

Des de la Raspberry, prova:

```
curl -X POST "https://api.telegram.org/bot<TOKEN>/sendMessage" \
  -d "chat_id=<CHAT_ID>" \
  -d "text=Hola des del BernatLab!"
```

Si tot va bé, rebràs el missatge al grup.

66.7 Com automatitzar des de la RPi

Crea un petit script Python que envia missatges:

Crea `~/homelab/scripts/telegram-notify.py`:

```
#!/usr/bin/env python3
"""Envia un missatge a Telegram."""
import os
import sys
import requests

TOKEN = os.environ.get("TELEGRAM_TOKEN")
CHAT_ID = os.environ.get("TELEGRAM_CHAT_ID")

if not TOKEN or not CHAT_ID:
    print("Falten TELEGRAM_TOKEN o TELEGRAM_CHAT_ID", file=sys.stderr)
    sys.exit(1)

if len(sys.argv) < 2:
    print("Ús: telegram-notify.py <missatge>", file=sys.stderr)
    sys.exit(1)

text = " ".join(sys.argv[1:])

r = requests.post(
```

```

    f"https://api.telegram.org/bot{TOKEN}/sendMessage",
    json={"chat_id": CHAT_ID, "text": text, "parse_mode": "HTML"},
    timeout=10,
)
r.raise_for_status()
print("Enviat.")

```

Crea `~/homelab/secrets/telegram.env`:

```

TELEGRAM_TOKEN=1234567890:ABCdefGHIjklMNOpqrsTUVwxyz
TELEGRAM_CHAT_ID=-1001234567890

```

Fes-lo executable:

```
chmod +x ~/homelab/scripts/telegram-notify.py
```

Prova'l:

```

source ~/homelab/secrets/telegram.env
~/homelab/scripts/telegram-notify.py "Prova de Telegram"

```

66.8 Integrar amb Uptime Kuma

Recordes el **Cap 62**? Uptime Kuma ja permet enviar alertes per Telegram. Si ho vas configurar, ja ho tens.

Si no, ara és el moment. A Uptime Kuma:

1. Settings → Notifications → Telegram.
2. Activa.
3. Posa el bot token.
4. Posa el chat_id.
5. Fes clic a "Test".
6. Aplica a tots els monitors que vulguis.

66.9 Integrar amb Node-RED

A Node-RED, podem afegir un node de Telegram:

1. **Manage palette** → Install → busca `node-red-contrib-telegrambot`.
2. Després d'instal·lar, tindràs nodes `telegram bot`, `telegram sender`, `telegram receiver`, `telegram command`, `telegram event`.
3. Configura el bot: - **Bot username**: el que vas triar. - **Bot token**: el token.
4. Usa el node `telegram sender` per enviar missatges.

Exemple: una automatització que envia una alerta quan la temperatura baixa de 5°C.

1. Node `mqtt in` a `sensors/hort1/temperatura`.
2. Node `function` que filtra temperatures baixes.
3. Node `telegram sender` configurat amb el bot.
4. Connecta'ls.

Deploy. Prova publicant una temperatura baixa a MQTT. Rebràs un missatge al grup de Telegram.

66.10 Com enviar missatges amb format

Telegram accepta HTML i Markdown. Pots fer missatges bonics:

```
text = ""
<b>■■ Alerta BernatLab</b>

<b>Node:</b> hort1
<b>Sensor:</b> temperatura
<b>Valor:</b> 2.3°C
<b>Llindar:</b> 5°C
<b>Missatge:</b> Risc de gelada!
""
```

Per usar HTML, posa `parse_mode=HTML` a la crida a l'API.

Per Markdown, `parse_mode=MarkdownV2`.

66.11 Comandes del bot

Pots fer que el bot respongui a comandes. Per exemple, `/status` retorna l'estat dels serveis.

A Node-RED, amb el node `telegram command`:

1. Configura `/status` com a command.
2. Connecta'l a un node `function` que construeixi el missatge.
3. Connecta'l a un node `telegram sender`.

Ara quan algú escrigui `/status` al grup, rebrà l'estat actual.

66.12 Bones pràctiques

1. **Un sol grup per a tot.** No creïs 10 grups.
2. **No saturis.** Limita les alertes a les importants.
3. **Usa emojis** per identificar ràpidament la severitat: ■ critical, ■■ warning, ■■■ info.
4. **Inclou l'acció a prendre.** No només "s'ha trencat X", sinó "s'ha trencat X, fer Y per arreglar-ho".
5. **Limita el volum.** Si reps 100 missatges al dia, ja no els llegeixes.
6. **No comparteixis el token.** És com una contrasenya.

66.13 Què ve després

Ja tens alertes al mòbil. Al **Cap 67** afegirem **Prometheus + Alertmanager** per a monitoratge avançat, i al **Cap 68** aprendrem a fer **runbooks** per als incidents.

66.14 Errors habituals

Error 1: el bot no envia res.

Comprova el token i el chat_id. Mira els logs de Node-RED o del script Python.

Error 2: el bot és al grup però no rep missatges.

Això passa si el grup té "anti-spam" molt estricte. Assegura't que el bot té permís per llegir i enviar.

Error 3: el missatge s'envia però no el reps.

Comprova que tens notificacions actives per a aquest grup al teu mòbil.

Error 4: format HTML invàlid.

Si el missatge té un < sense tancar, Telegram el rebutja. Escapa caràcters especials.

66.15 Resum

Telegram és la manera més còmoda de rebre alertes del BernatLab. Hem vist:

- Crear el bot.
- Configurar el grup.
- Enviar missatges des de la RPi.
- Integrar amb Uptime Kuma.
- Integrar amb Node-RED.
- Comandes interactives.

66.16 Exercicis pràctics

1. Crea el bot de Telegram.
2. Crea el grup i afegeix-hi el bot.
3. Envia un missatge des de la RPi amb `curl`.
4. Configura Uptime Kuma per enviar alertes al grup.
5. Configura Node-RED per enviar alertes quan la temperatura baixa de 5°C.
6. Afegeix una comanda `/status` al bot.
7. Documenta la configuració al `home lab/setup-log.md`.

Capitol 67 — Prometheus i alertes avançades

*“Uptime Kuma et diu si un servei està viu o mort. Prometheus et diu com està de salut. La diferència és subtil però fonamental.”**

67.1 Què aprendràs

- Què és Prometheus i per què serveix.
- Com instal·lar Prometheus + Node Exporter + cAdvisor + Alertmanager.
- Com crear mètriques personalitzades.
- Com configurar alertes intel·ligents.
- Com integrar-ho amb Telegram.
- Com visualitzar les mètriques a Grafana.

67.2 Durada estimada

1-1.5 hores.

67.3 Què és Prometheus

Prometheus és un sistema de monitoratge de mètriques (números) amb:

- **Pull**: ell mateix demana les mètriques als serveis cada X segons.
- **Series temporals**: emmagatzema cada mètrica amb un timestamp.
- **Alerting**: té el seu propi sistema d'alertes (Alertmanager).
- **Lenguatge de consultes**: PromQL, potent i flexible.

Combinat amb **Grafana** (que ja tens) tens una solució completa.

67.4 Arquitectura

```
[ Node Exporter ]   ■■■
[ cAdvisor ]       ■■■
[ Mosquitto exporter ]■■■■→ [ Prometheus ] ■■→ [ Alertmanager ] ■■→ [ Telegram ]
[ Custom exporter ] ■■■                ■■→ [ Grafana ]
```

Cada component:

- **Node Exporter**: mètriques del sistema (CPU, RAM, disc, xarxa, temperatura).
- **cAdvisor**: mètriques dels contenidors Docker.
- **Mosquitto exporter**: mètriques del broker MQTT.
- **Custom exporter**: mètriques específiques (per exemple, l'estat del node LoRa).

- **Prometheus:** recull tot i emmagatzema.
- **Alertmanager:** gestiona les alertes.
- **Grafana:** visualitza.

67.5 Instal·lació

Crea ~/home/lab/compose/prometheus.yml:

```
version: "3.8"

services:
  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    restart: unless-stopped
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
      - ./prometheus/rules.yml:/etc/prometheus/rules.yml
      - ./data/prometheus:/prometheus
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--storage.tsdb.path=/prometheus'
      - '--storage.tsdb.retention.time=30d'
    ports:
      - "9090:9090"

  alertmanager:
    image: prom/alertmanager:latest
    container_name: alertmanager
    restart: unless-stopped
    volumes:
      - ./prometheus/alertmanager.yml:/etc/alertmanager/alertmanager.yml
    command:
      - '--config.file=/etc/alertmanager/alertmanager.yml'
    ports:
      - "9093:9093"

  node-exporter:
    image: prom/node-exporter:latest
    container_name: node-exporter
    restart: unless-stopped
    pid: host
    volumes:
      - /proc:/host/proc:ro
      - /sys:/host/sys:ro
      - /:/rootfs:ro
    command:
      - '--path.procfs=/host/proc'
      - '--path.sysfs=/host/sys'
      - '--path.rootfs=/rootfs'
    ports:
      - "9100:9100"

  cadvisor:
    image: gcr.io/cadvisor/cadvisor:latest
    container_name: cadvisor
    restart: unless-stopped
    privileged: true
    volumes:
      - /:/rootfs:ro
```

```

- /var/run:/var/run:ro
- /sys:/sys:ro
- /var/lib/docker:/var/lib/docker:ro
ports:
- "8080:8080"

```

Crea ~/homelab/compose/prometheus/prometheus.yml:

```

global:
  scrape_interval: 30s
  evaluation_interval: 30s

rule_files:
- "rules.yml"

alerting:
  alertmanagers:
  - static_configs:
    - targets: ['alertmanager:9093']

scrape_configs:
- job_name: prometheus
  static_configs:
  - targets: ['localhost:9090']

- job_name: node
  static_configs:
  - targets: ['node-exporter:9100']

- job_name: cadvisor
  static_configs:
  - targets: ['cadvisor:8080']

```

Crea ~/homelab/compose/prometheus/rules.yml:

```

groups:
- name: bernatlab
  rules:
  - alert: HighCPUUsage
    expr: 100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 90
    for: 5m
    labels:
      severity: warning
    annotations:
      summary: "CPU alta a {{ $labels.instance }}"
      description: "CPU al {{ $value }}% durant més de 5 minuts."

  - alert: LowMemory
    expr: (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100 < 10
    for: 5m
    labels:
      severity: critical
    annotations:
      summary: "RAM baixa a {{ $labels.instance }}"

  - alert: DiskFull
    expr: (node_filesystem_avail_bytes{fstype!="tmpfs"} / node_filesystem_size_bytes) * 100 < 10
    for: 10m
    labels:
      severity: critical
    annotations:
      summary: "Disc ple a {{ $labels.instance }}"

  - alert: ContainerDown

```

```

    expr: up{job=~"cadvisor|prometheus"} == 0
    for: 2m
    labels:
      severity: critical
    annotations:
      summary: "Contenidor {{ $labels.instance }} caigut"

```

Crea `~/homelab/compose/prometheus/alertmanager.yml`:

```

global:
  resolve_timeout: 5m

route:
  group_by: ['alertname', 'instance']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  receiver: 'telegram'

receivers:
- name: 'telegram'
  webhook_configs:
  - url: 'http://alert-webhook:8080/alerts'

```

Crea `~/homelab/compose/alert-webhook/`:

```

# app.py
import os, requests
from flask import Flask, request

app = Flask(__name__)

@app.route("/alerts", methods=["POST"])
def alerts():
    data = request.json
    for alert in data.get("alerts", []):
        status = alert.get("status", "unknown")
        emoji = "■" if status != "resolved" else "■"
        text = alert.get("annotations", {}).get("summary", "")
        desc = alert.get("annotations", {}).get("description", "")
        msg = f"{emoji} {text}\n{desc}"
        requests.post(
            f"https://api.telegram.org/bot{os.environ['TELEGRAM_TOKEN']}/sendMessage",
            json={"chat_id": os.environ["TELEGRAM_CHAT_ID"], "text": msg},
        )
    return "OK", 200

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)

```

Engega:

```

cd ~/homelab/compose
docker compose -f prometheus.yml up -d

```

Afegeix el webhook al compose.

67.6 Verificar

- Prometheus UI: <http://hortosona:9090>
- Alertmanager UI: <http://hortosona:9093>
- Node Exporter: <http://hortosona:9100/metrics>

A Prometheus, vés a **Status** → **Targets** i comprova que tots els jobs estan "UP".

67.7 Connectar Grafana a Prometheus

A Grafana:

1. Connections → Data sources → Add data source.
2. Tria **Prometheus**.
3. URL: `http://prometheus:9090`.
4. Save & test.

Ara pots crear panells que combinen dades de Grafana (InfluxDB) i Prometheus (CPU, RAM, etc.).

67.8 Dashboard recomanat

Importa el dashboard oficial de Node Exporter:

1. Grafana → + → **Import**.
2. Enganxa l'ID 1860 (Node Exporter Full).
3. Tria el data source Prometheus.
4. Import.

Ara tens un dashboard complet amb CPU, RAM, disc, xarxa, processsos, etc.

67.9 Provar les alertes

Per forçar una alerta, podem aturar un servei:

```
docker stop grafana
```

Espera 2-3 minuts. Hauries de rebre un missatge a Telegram: "Contenedor grafana caigut".

Torna a engegar:

```
docker start grafana
```

Rebràs "Resolved".

67.10 Mètriques personalitzades: l'estat del node LoRa

Podem exposar l'estat del node LoRa com a mètriques. Per exemple, un script Python que exposa "l'última vegada que el node ha enviat dades":

```
# lora-exporter.py
from prometheus_client import start_http_server, Gauge
import paho.mqtt.client as mqtt
import time
import os
```

```
last_seen = Gauge('lora_node_last_seen_timestamp', 'Última vegada que el node ha enviat dades', ['node'])
```

```
def on_message(client, userdata, msg):
    import time
    last_seen.labels(node="hort1").set(time.time())
```

```
client = mqtt.Client()
client.username_pw_set("bernat", os.environ["MQTT_PASSWORD"])
client.connect("mosquitto", 1883, 60)
client.subscribe("sensors/#")
client.on_message = on_message

start_http_server(9119)
client.loop_forever()
```

Afegeix aquest servei al compose de Prometheus. Ara pots crear una alerta:

```
- alert: LoRaNodeOffline
  expr: time() - lora_node_last_seen_timestamp{node="hort1"} > 3600
  for: 10m
  labels:
    severity: warning
  annotations:
    summary: "Node hort1 sense dades des de fa més d'1h"
```

67.11 Què ve després

Ja tens monitoratge complet. Al **Cap 68** aprendrem a fer **runbooks** per reaccionar ràpidament als incidents. Al **Cap 69** farem el **DRP** amb un test real.

67.12 Errors habituals

Error 1: Prometheus no recull mètriques.

Mira els targets a la UI. Si un job està "DOWN", comprova la URL i la xarxa.

Error 2: les alertes no s'envien.

Mira els logs d'Alertmanager. Comprova la configuració del webhook.

Error 3: les alertes s'envien massa.

Ajusta els llindars i els temps **for** : **5m**. Sigues estricte.

67.13 Resum

Prometheus + Alertmanager és la combinació de monitoratge avançat més potent per a un homelab. Hem vist:

- Arquitectura completa.
- Instal·lació amb Compose.
- Regles d'alerta bàsiques.
- Integració amb Telegram.
- Mètriques personalitzades.
- Dashboard a Grafana.

67.14 Exercicis pràctics

1. Instal·la Prometheus + Alertmanager + Node Exporter + cAdvisor.

2. Verifica que els targets són "UP".
3. Importa el dashboard de Node Exporter (ID 1860).
4. Configura alertes per Telegram.
5. Crea una alerta personalitzada (per exemple, "Node LoRa offline").
6. Força una alerta aturant un servei.
7. Documenta la configuració.

Capitol 68 — Runbooks: quan falla alguna cosa

"Un runbook és la diferència entre resoldre un incident en 5 minuts o en 5 hores."

68.1 Què aprendràs

- Què és un runbook i quan usar-lo.
- Com estructurar un runbook.
- Com escriure els teus primers 3 runbooks.
- On desar-los.
- Com millorar-los amb el temps.

68.2 Durada estimada

30-45 minuts.

68.3 Per què runbooks

Ara tens un sistema amb molts components: Docker, Portainer, Uptime Kuma, MQTT, InfluxDB, Grafana, Node-RED, Telegram, Prometheus, Alertmanager, node LoRa. Si un falla, has de saber què fer.

Sense un runbook:

- Tens memòria (que falla).
- Has de cercar a Internet cada vegada.
- El pànic et porta a cometre errors.

Amb un runbook:

- Tens les passes exactes.
- Les executes sense pensar.
- El temps de resolució baixa.

Un runbook **es crea quan resols un incident**, no pas abans. Ara tens un sistema relativament estable — bona oportunitat per escriure els primers.

68.4 Estructura d'un runbook

```
# Runbook: [Títol del problema]
```

```
## Síntomes
```

```
- Què veus.
```

```
## Diagnòstic
```

```
1. Comandes per confirmar.
```

```
## Solució
```

```
1. Passes per resoldre.
```

```
## Verificació
```

```
- Com saber que s'ha resolt.
```

```
## Prevenció
```

```
- Com evitar-ho en el futur.
```

```
## Contactes
```

```
- Qui avisar.
```

```
## Última actualització
```

```
- Data i canvis.
```

68.5 On desar els runbooks

A la teva estructura:

```
homelab/
├── runbooks/
│   ├── README.md          # índex
│   ├── 001-portainer-down.md
│   ├── 002-grafana-no-data.md
│   ├── 003-lora-offline.md
│   └── ...
```

Crea la carpeta:

```
mkdir -p ~/homelab/runbooks
```

Crea un **README.md** índex:

```
# Runbooks del BernatLab

## Incidents
- [001](001-portainer-down.md) – Portainer no respon
- [002](002-grafana-no-data.md) – Grafana no mostra dades
- [003](003-lora-offline.md) – Node LoRa offline
- [004](004-mqtt-disconnected.md) – Mosquitto no accepta connexions

## Manteniment
- [101](101-update-containers.md) – Actualitzar contenidors
- [102](102-rotate-secrets.md) – Rotar secrets

## Recuperació
- [201](201-restore-raspberry.md) – Restaurar Raspberry
- [202](202-data-loss.md) – Pèrdua de dades
```

Versiona aquesta carpeta a Git. Així tens historial de canvis.

68.6 Runbook 001: Portainer no respon

Crea ~/homelab/runbooks/001-portainer-down.md:

```
# Runbook 001: Portainer no respon

## Síntomes
- El navegador no pot connectar a `https://hortosona:9443`.
- Altres serveis funcionen normalment.

## Diagnòstic

1. Comprova l'estat del contenidor:

docker ps | grep portainer ```

1. Si el contenidor està aturat, mira els logs: ```bash docker logs portainer --tail 100 ```

1. Si està en marxa, comprova el port: ```bash ss -tulnp | grep 9443 ```
```

Solució

Si el contenidor està aturat:

1. Reiniciar: ```bash docker start portainer ```
1. Esperar 30 segons.
1. Comprovar: ```bash curl -k https://localhost:9443 ```

Si el contenidor està en marxa però no respon:

1. Reiniciar: ```bash docker restart portainer ```
1. Si continua, mirar logs per errors: ```bash docker logs portainer --tail 200 ```
1. Si hi ha error de base de dades: ```bash docker stop portainer cp -r ~/homelab/compose/data/portainer.bak/* \ ~/homelab/compose/data/portainer/ docker start portainer ```

Verificació

- `curl -k https://localhost:9443` retorna algun HTTP status.
- El navegador pot connectar i fer login.

Prevenició

- Mantenir espai en disc (Portainer necessita 1 GB lliure).
- Comprovar memòria disponible (mínim 256 MB).
- Configurar `restart: unless-stopped` (ja ho tens al compose).

Contactes

- @bernat a Telegram

Última actualització

- 2026-07-09 per Bernat

68.7 Runbook 002: Grafana no mostra dades

Runbook 002: Grafana no mostra dades noves

Síntomes

- Les gràfiques estan buides o no s'actualitzen.
- Les dades antigues es veuen, les noves no.

Diagnòstic

1. Comprovar que InfluxDB rep dades: ````bash docker exec influxdb influx query 'from(bucket:"hort") |> range(start:-10m)' ````
1. Si no hi ha dades, comprovar Telegraf: ````bash docker logs telegraf --tail 50 ````
1. Si Telegraf no es connecta, comprovar Mosquitto: ````bash mosquitto_sub -h localhost -p 1883 -u bernat -P 'contrasenya' -t 'sensors/#' -v ````

Solució

Si Mosquitto no funciona:

1. Reiniciar: ````bash docker restart mosquitto ````
1. Esperar 30 segons.
1. Provar de nou amb `mosquitto_sub`.

Si Telegraf no es connecta:

1. Reiniciar: ````bash docker restart telegraf ````
1. Esperar 30 segons.
1. Comprovar logs: ````bash docker logs telegraf --tail 50 ````

Si Grafana no es connecta a InfluxDB:

1. Anar a Grafana → Connections → Data sources.
2. Editar el data source InfluxDB.
3. Verificar URL, token, org, bucket.

4. Save & test.

Verificació

- Publicar una temperatura de prova a `sensors/hort1/temperatura`.
- Esperar 1 minut.
- La gràfica de Grafana ha de mostrar el punt.

Prevenció

- Monitorar els serveis amb Uptime Kuma.
- Configurar alertes a Prometheus per "no data".
- Revisar logs periòdicament.

Contactes

- @bernat a Telegram

Última actualització

- 2026-07-09 per Bernat

68.8 Runbook 003: Node LoRa offline

Runbook 003: Node LoRa no envia dades

Síntomes

- El node hauria d'enviar dades cada 5-15 minuts.
- Fa més d'1 hora que no apareixen dades noves a Grafana.
- Alerta de Prometheus o Uptime Kuma.

Diagnòstic

1. Comprovar que el node té bateria (si té placa solar, comprovar-la).
 2. Si tens accés físic, mirar el LED d'estat.
 3. Publicar un missatge manual a MQTT per verificar la cadena:

```
```bash mosquitto_pub -h hortosona -p 1883 -u bernat -P 'contrasenya' \ -t 'sensors/hort1/temperatura' \ -m '{"node":"hort1","sensor":"temperatura","value":99.9,"unit":"C"}' ```
```
1. Si el missatge arriba a Grafana, el problema és del node, no del servidor.

## Solució

### Si és un node amb bateria:

1. Anar al camp.
2. Obrir la caixa.
3. Comprovar la bateria.
4. Si cal, substituir-la o recarregar-la.
5. Reiniciar el node (desendollar i tornar a endollar).

### Si és un node amb placa solar:

1. Comprovar que la placa està neta.
2. Comprovar que la bateria es carrega.
3. Mirar si la placa té ombra (arbres nous?).

### Si el node sembla correcte però no envia:

1. Comprovar la cobertura LoRa.
2. Verificar que el gateway està en marxa.
3. Reiniciar el node i el gateway.

## Verificació

- Esperar 15-30 minuts.
- Hauries de veure dades noves a Grafana.

## Prevenció

- Usar bateries de més capacitat.
- Afegir un panell solar més gran.
- Configurar deep sleep agressiu.
- Posar el node en un lloc amb bona cobertura.

## Contactes

- @bernat a Telegram

## Última actualització

- 2026-07-09 per Bernat

## 68.9 Com millorar els runbooks

Cada vegada que resols un incident:

1. Escribe el runbook si no existeix.
2. Actualitza'l si existeix.
3. Prova'l la propera vegada.
4. Comparteix-lo si pot ajudar algú.

Al cap d'un any, tindràs 10-20 runbooks que cobreixen el 90% dels incidents habituals. Això et permetrà

### ## 68.10 Com provar els runbooks

Cada trimestre, dedica una hora a:

1. Tria un runbook.
2. Llegeix-lo.
3. Executa'l en un entorn de test (no en producció).
4. Comprova que funciona.
5. Corregeix el que no.

Això t'assegura que els runbooks estan al dia.

### ## 68.11 Què ve després

Ja tens runbooks. Al **Cap 69** farem el **DRP** (Disaster Recovery Plan) amb un test real: simular una t

### ## 68.12 Errors habituals

**Error 1: runbooks massa genèrics**.

"No facis res" no és un runbook. Ha de tenir comandes específiques.

**Error 2: runbooks desactualitzats**.

Si canvies el sistema, els runbooks queden obsolets. Revisa'ls cada mes.

**Error 3: runbooks que ningú llegeix**.

Guarda'ls a un lloc accessible. Referencia'ls a les alertes.

**Error 4: no provar els runbooks**.

Si no els has provat, no saps si funcionen.

### ## 68.13 Resum

Els runbooks et fan estalviar temps i estrès. Hem vist:

- Què és un runbook.
- Com estructurar-lo.
- Tres runbooks d'exemple (Portainer, Grafana, LoRa).
- On desar-los.
- Com millorar-los.

### ## 68.14 Exercicis pràctics

1. Crea la carpeta `~/homelab/runbooks/``.
2. Escribe 3 runbooks basats en els 3 exemples d'aquest capítol.
3. Adaptals al teu sistema (canviar noms, paths, etc.).
4. Crea un `README.md`` índex.
5. Versionals a Git.
6. Prova un runbook amb una situació simulada.

# Capítol 69 — DRP: el dia que es crema tot

*""Les còpies sense un pla de recuperació són diners al calaix. Aquest capítol és el pla.""\**

## 69.1 Què aprendràs

- Què és un DRP i per què és vital.
- Com fer un test real de recuperació.
- Quines decisions has pres abans del desastre.
- Com restaurar el sistema pas a pas.
- Què fer si el hardware falla.

## 69.2 Durada estimada

2-4 hores (comptant el test real).

## 69.3 Per què un test real

A la majoria d'homelabs, el DRP és un document que ningú ha provat mai. Això és perillós perquè:

- El document pot tenir errors.
- Pot ser obsolet.
- Pot no funcionar en la pràctica.

La solució és **fer un test real**, en un entorn segur, **abans** que passi el desastre de veritat.

En aquest capítol farem un test de restauració completa. **Apagaràs la Raspberry, borraràs la microSD, i la restauraràs** des de zero amb les còpies que tens.

Això et portarà 1-2 hores. Però et donarà la confiança que el teu sistema és recuperable.

## 69.4 Què necessites per al test

- Una **microSD de recanvi** (32-64 GB).
- Un **lector de microSD USB**.
- Un **PC** amb balenaEtcher o similar.
- La **còpia de seguretat de la microSD** (la del cap 60).
- Les **còpies de restic** (a un disc extern o al núvol).
- 1-2 hores de temps lliure.



## 69.5 Preparació: abans del test

1. **Assegura't que les còpies estan al dia:** ````bash restic -r ~/backups/bernatlab snapshots ```` La còpia més recent ha de ser de fa menys de 24 h.
1. **Comprova la integritat de les còpies:** ````bash restic -r ~/backups/bernatlab check ````
1. **Tingues a mà la llista de serveis i les seves configuracions:** - `~/homelab/compose/` - `~/homelab/secrets/` - `~/homelab/data/`
1. **Documenta l'estat actual:** - Quins contenidors estan corrent? - Quines són les IPs? - Quins serveis són crítics?
1. **Avisa la família o la gent que depèn del sistema** que el sistema estarà inactiu durant 1-2 hores.

## 69.6 El test: pas a pas

### Pas 1: atura tots els serveis ordenadament

```
cd ~/homelab/compose
docker compose -f portainer.yml down
docker compose -f uptime-kuma.yml down
docker compose -f mqtt.yml down
docker compose -f influxdb.yml down
docker compose -f grafana.yml down
docker compose -f node-red.yml down
docker compose -f telegraf.yml down
docker compose -f prometheus.yml down
```

Així tanques tot netament. Si hi ha algun contenidor amb problema, pots forçar-lo:

```
docker stop $(docker ps -aq)
```

### Pas 2: apaga la Raspberry

```
sudo shutdown -h now
```

Espera 30 segons. La LED verda parpalleja 10 vegades i s'apaga.

### Pas 3: treu la microSD

Desendolla la RPi. Treu la microSD.

### Pas 4: simula la pèrdua total

Aquí ve la part interessant. Tens dues opcions:

#### **Opció A: destruir la microSD "dolenta" (recomanada).**

Si tens una microSD de recanvi, fes servir aquesta per al test. La "dolenta" la guardes com a testimoni de la pèrdua.

#### **Opció B: formatar la microSD "dolenta".**

Si vols reutilitzar-la:

Al teu PC amb lector de microSD:

- **Windows:** usa **SD Card Formatter** o l'eina de disc.

- **Mac:** `diskutil eraseDisk FAT32 NOOBS /dev/disk2` (canvia disk2 pel teu).
- **Linux:** `sudo mkfs.vfat -F 32 /dev/sdb1` (canvia sdb1 pel teu).

## Pas 5: flasheja la microSD amb la imatge de recuperació

Agafes la còpia de la microSD (la que vas fer al cap 60) i la flasheges a la nova microSD:

Al teu PC:

- **balenaEtcher:** selecciona la imatge, selecciona la microSD, flash.
- **Raspberry Pi Imager:** igual.

## Pas 6: inserir la microSD i arrencar

Insereix la nova microSD a la RPi, endolla-la, i engega-la.

La RPi hauria d'arrencar exactament com estava abans del test. Verifica:

- Tens accés per SSH?
- Tens xarxa?
- Tens les teves dades?

## Pas 7: comprovar serveis

```
docker ps
```

Tots els serveis haurien d'estar corrent. Si no, engega'ls:

```
cd ~/homelab/compose
for f in *.yaml; do docker compose -f "$f" up -d; done
```

## Pas 8: verificar la integritat

Per a cada servei, comprova que funciona:

- Portainer: <https://hortosona:9443>
- Uptime Kuma: <http://hortosona:3001>
- Grafana: <http://hortosona:3000>
- Node-RED: <http://hortosona:1880>

Si tot funciona, **el test ha estat un èxit**.

## Pas 9: documentar el temps

Anota quant ha durat cada pas. Això t'ajudarà a calcular el teu RTO (Recovery Time Objective).

Exemple:

- Apagar serveis: 2 min
- Apagar Raspberry: 1 min
- Treure microSD: 1 min
- Flashear imatge: 10 min

- Arrencar: 2 min
- Verificar serveis: 10 min
- **Total: 26 min**

Si el teu RTO és 4 h, estàs molt bé. Si trigues més, cal optimitzar.

## 69.7 Test alternatiu: restauració de dades amb restic

Aquest test és menys dramàtic però igual d'important. Simula que has esborrat un fitxer per error.

1. Esborra un fitxer qualsevol (no crític): ````bash rm ~/homelab/compose/test-file.txt ````

1. Troba'l a la còpia: ````bash restic -r ~/backups/bernatlab find test-file.txt ````

1. Restaura'l: ````bash restic -r ~/backups/bernatlab restore latest \ --target /tmp/restored \ --include /home/bernat/homelab/compose/test-file.txt ````

1. Comprova: ````bash cat /tmp/restored/home/bernat/homelab/compose/test-file.txt ````

Si funciona, les teves còpies són vàlides.

## 69.8 Què fer si el hardware falla

Si la RPi falla (cosa que passarà), tens opcions:

### Opció A: comprar una RPi nova

- Preu: 50-100 € (RPi 4 o 5).
- Restaurar la microSD antiga (si la RPi ha fallat però la microSD està bé).
- O flashear una nova amb la imatge de còpia.

### Opció B: usar un mini PC

Si la RPi no està disponible, pots muntar el sistema en un mini PC amb Debian. La majoria de coses funcionaran igual.

### Opció C: usar el núvol temporalment

Si necessites el sistema actiu ràpidament, pots contractar un VPS i restaurar-hi les dades.

## 69.9 El DRP documentat

Un cop fet el test, documenta el DRP:

```
DRP del BernatLab

RTO: 1 hora
RPO: 24 hores (còpies diàries)

Escenari 1: Fallada de la Raspberry

Detecció: Uptime Kuma, Telegram.
Responsable: Bernat.
Temps de recuperació: 30 min - 2 h.
```

Passes:

1. Identificar la fallada.
2. Si és de hardware, comprar Raspberry nova.
3. Flashear la microSD amb la imatge de còpia.
4. Inserir a la Raspberry i arrencar.
5. Verificar que tots els serveis s'han reiniciat.
6. Verificar que les dades estan intactes (recollir una dada de test).

## Escenari 2: Pèrdua de dades

**\*\*Detecció\*\***: Alerta o descoberta accidental.

**\*\*Responsable\*\***: Bernat.

**\*\*Temps de recuperació\*\***: 10-30 min.

Passes:

1. Identificar què s'ha perdut.
2. Buscar a les còpies de restic.
3. Restaurar amb `restic restore`.
4. Verificar que el fitxer és correcte.

## Escenari 3: Atac de seguretat

**\*\*Detecció\*\***: Alerta de seguretat o comportament estrany.

**\*\*Responsable\*\***: Bernat.

**\*\*Temps de recuperació\*\***: 2-4 h.

Passes:

1. Aïllar la Raspberry de la xarxa.
2. Identificar el vector d'atac.
3. Restaurar des d'una còpia neta.
4. Rotar tots els secrets.
5. Investigar i documentar.

## Escenari 4: Desastre natural

**\*\*Detecció\*\***: Pèrdua total del hardware.

**\*\*Responsable\*\***: Bernat.

**\*\*Temps de recuperació\*\***: 1-7 dies (depèn del hardware).

Passes:

1. Comprar hardware nou.
2. Restaurar des de còpia al núvol.
3. Reconfigurar segons el README.

## Recursos

- Còpies al núvol: b2:bernatlab-backups
- Imatges de microSD: a la caixa forta / NAS d'un familiar
- Codi i configuració: <https://github.com/BernatMora/bernatlab>
- Recovery codes Tailscale: a Bitwarden

## Última actualització

- 2026-07-09 per Bernat (després del primer test)

## 69.10 Què fer després del test

Si el test ha anat bé:

- Documenta'l.
- Desa la documentació al repo.
- Comparteix el DRP amb algú de confiança (la teva parella, un amic).

Si ha fallat:

- Identifica on ha fallat.
- Corregeix la documentació.
- Torna-ho a provar.
- No t'amoïnis — un DRP és un document viu.

## 69.11 Resum final del Mòdul 7

Acabes de completar el camí de tenir una Raspberry en una caixa a tenir un **sistema professional** d'IoT per a l'hort. Has après:

- A instal·lar i endureir una Raspberry Pi.
- A muntar una cadena de dades completa.
- A rebre alertes al mòbil.
- A monitorar tot el sistema.
- A reaccionar als incidents amb runbooks.
- A recuperar el sistema si passa el pitjor.

Això és molt. La majoria d'homelabs no arriben ni a la meitat d'això.

## 69.12 Què ve després

Ara tens un BernatLab funcional. Les possibles ampliacions són infinites:

- Més nodes LoRa.
- Nous sensors (vent, pluja, radiació solar).
- Càmera amb visió per computador.
- Més serveis (Nextcloud per a fitxers, Immich per a fotos, etc.).
- Domini propi i HTTPS.
- Un assistent d'IA local.

Tots aquests temes es poden convertir en futurs mòduls del llibre.

## 69.13 Agraïments

Si has arribat fins aquí, has après molt. Jo també he après escrivint-ho.

Aquest llibre és un document viu. Si trobes errors, si canvies alguna cosa al teu sistema, si tens una idea nova, torna a editar-lo. El coneixement només és útil si està actualitzat.

## 69.14 Exercicis pràctics

1. Fes una còpia de la microSD si no la tens.
2. Programa un test de DRP un dia que tinguis temps.

3. Documenta el test al `home\lab\postmortems/`.
4. Escriu el DRP al `home\lab\DRP.md`.
5. Comparteix-lo amb algú de confiança.
6. Repeteix el test cada 6 mesos.